Flash Exchange — Detailed Design

Low-latency spot currency exchange: Java matching engine, Aeron messaging, LMAX Disruptor, real-time web trading platform.

Java 21 | Aeron IPC / UDP | Aeron Cluster | Disruptor | SBE | Netty WebSockets | React + TypeScript | PostgreSQL | Redis | Primary / hot standby

Flash Exchange — Detailed Design

Status: **Draft v0.1 (design only, no implementation)** · Scope: **Spot trading** · Date: 2026-11-03

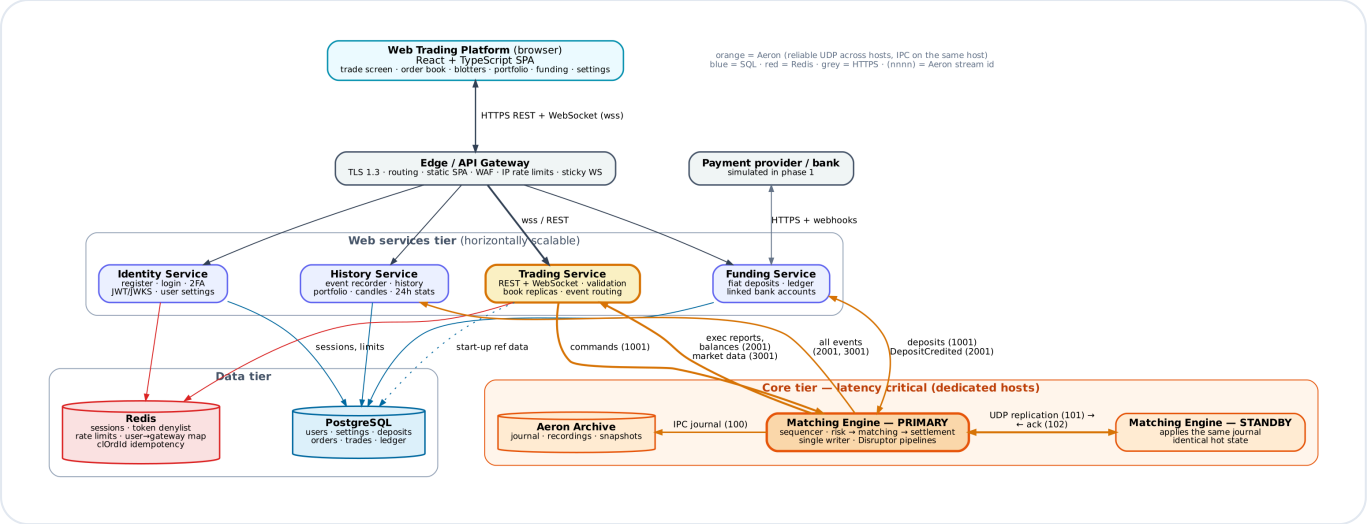
Flash Exchange is a spot currency exchange. Registered users fund their accounts from fiat, place **limit** and **market** orders (Good-Till-Cancel by default), see a live order book, blotters and portfolio, and cancel orders. The core is a low-latency **Java matching engine**; all backend messaging is **Aeron** (IPC on-box, reliable UDP across boxes) and in-process concurrency uses the **LMAX Disruptor**.

Document map

#	Document	What it covers
1	Architecture	System context, component diagram, deployment topology, tech stack, key decisions (ADRs)
2	Matching Engine	Order book data structures, matching rules, order types/TIF, Disruptor pipeline, risk, determinism, recovery
3	Messaging & Protocol	Aeron channels/stream IDs, SBE message catalogue, sequencing & gap recovery
4	Trading Service	Order gateway, validation, session routing, market-data fan-out, WebSocket protocol
5	Web Platform	UI features, layout, state management, settings, auth flows
6	Accounts, Funding & Data	Ledger, deposits, balances/holds, PostgreSQL schema, persistence pipeline
7	Interactions	Sequence diagrams for every key flow, order state machine
8	NFRs, Operations & Roadmap	Latency/throughput targets, HA, security, observability, testing, delivery plan

Architecture diagrams are Graphviz (sources in `diagrams/*.dot`, rendered to PNG and SVG). Flow and sequence diagrams are Mermaid and render in GitHub, GitLab, VS Code and IntelliJ.

Executive summary



Components (by tier: Client → Edge → Web services → Core, plus Data)

1. **Matching Engine** (Java; PRIMARY + hot STANDBY) — owns order books *and* the authoritative trading balances (pre-trade funds hold + post-trade settlement) in one deterministic, single-writer state machine. Five internal layers: transport & sequencing → inbound Disruptor (journal ‖ replicate ‖ risk-hold → match → settle ‖ market data) → release gate & egress → in-memory state → control plane. Journal recorded in Aeron Archive (100), replicated to standby (101/102), snapshots; arbiter-driven failover with epoch fencing.
2. **Trading Service** (Java) — bridge between web and engine. Authenticates sessions, validates orders (static + soft balance check), forwards SBE commands over Aeron, routes acks/execution reports to the owning user, and fans out L2 order books / trades to browsers over WebSockets (snapshot + sequenced deltas, conflated).
3. **Identity Service** (Spring Boot) — registration, login, email verification, 2FA, JWT/refresh tokens, user settings (theme, colour scheme, defaults).
4. **Funding Service** (Spring Boot) — fiat deposits (payment-provider adapter, mocked in phase 1), double-entry ledger, sends `Deposit` commands to the Matching Engine to credit trading balances.
5. **History Service** (Java) — event recorder: subscribes to 2001/3001 (gaps filled from Archive replay 4001), writes orders/trades/balances history to PostgreSQL, serves history REST queries (trades blotter, order history, portfolio valuation, candles).
6. **Edge / API Gateway** (Envoy or Nginx) — TLS, WAF, routing, IP rate limits, sticky WebSockets, serves the SPA.
7. **Data tier** — **PostgreSQL** (users, ledger, history; a projection, not the source of truth) and **Redis** (sessions, token denylist, distributed rate limits, clOrdId idempotency, user→gateway map; never on the matching path).
8. **Web Platform** (React + TypeScript SPA) — responsive trading UI: order entry, live order book with own-order highlighting, open-orders & trades blotters, portfolio, deposits, settings, dark/light themes.

Key design decisions

- Fixed-point `long` prices/quantities everywhere; zero allocation on the hot path; SBE encoding; Agrona collections; object pools.
- **Single-writer per state**: one matching thread per instrument shard, one risk thread per user shard; no locks on the hot path.
- **Risk inside the Matching Engine** (not in Trading Service) so that funds holds and fills are serialised in one log — no double-spend races, no reconciliation loops. Trading Service still does a fast *soft* check for UX.
- **Determinism**: time is stamped once at sequencing and journaled, never read in business logic → exact replay, hot standby and audit.
- **Release gate**: no event leaves the engine until its command is journaled and acked by the standby → RPO 0.

- **Layered tiers**: Client → Edge → Web services → Core, with a Data tier (PostgreSQL + Redis) kept off the hot path.
- **Market orders** are IOC with a price-protection collar (default ±5% of best opposite) — never rest on the book.
- **TIF**: GTC (default, preselected), IOC, FOK; GTD reserved in the schema for later.
- **Market data to browsers**: L2 aggregated levels (price, total qty, order count), snapshot + sequenced incremental deltas, conflated to ≤ 20 updates/s per book per client; own-order highlighting computed client-side from the user's open orders (plus optional queue-position hint from Trading Service).

Additions to the original spec ("anything I missed")

Withdrawals; KYC/AML tiering; fee schedule (maker/taker) and fee reporting; price chart (OHLCV candles) and 24h ticker; public trade tape; self-trade prevention; post-only flag; price bands / circuit breakers and instrument halt; instrument reference data (tick size, lot size, min notional); idempotent client order IDs; cancel-all; rate limiting; 2FA, session/device management; audit log; admin/ops console (halt market, mass-cancel, user freeze); toasts/notifications for fills; CSV export of history; API keys for programmatic trading (later); reconciliation between ledger and engine balances; observability (latency histograms per hop).

Delivery plan (summary) — see 08

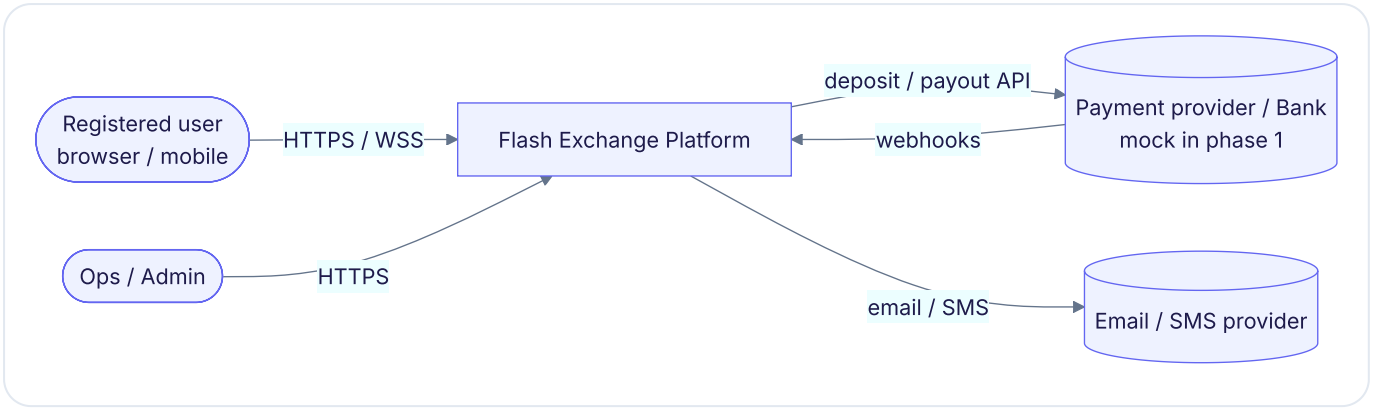
Phase	Goal
0 Foundations	Gradle multi-module repo, SBE schemas, Aeron media driver config, CI, test harness
1 Matching Engine	Order book + matching + risk + Disruptor pipeline, single node with Archive & snapshots; JMH + latency tests
2 Trading Service	Order gateway, validation, routing, market-data fan-out, WebSocket API
3 Identity, Funding, History	Auth, settings, deposits/ledger, history DB and query API
4 Web Platform	Full trading UI
5 Production hardening	Standby engine + replication + arbiter, failover drills, perf tuning, security, observability, load tests
6 Extensions	Withdrawals, KYC, API keys, admin console, GTD

Open questions for you

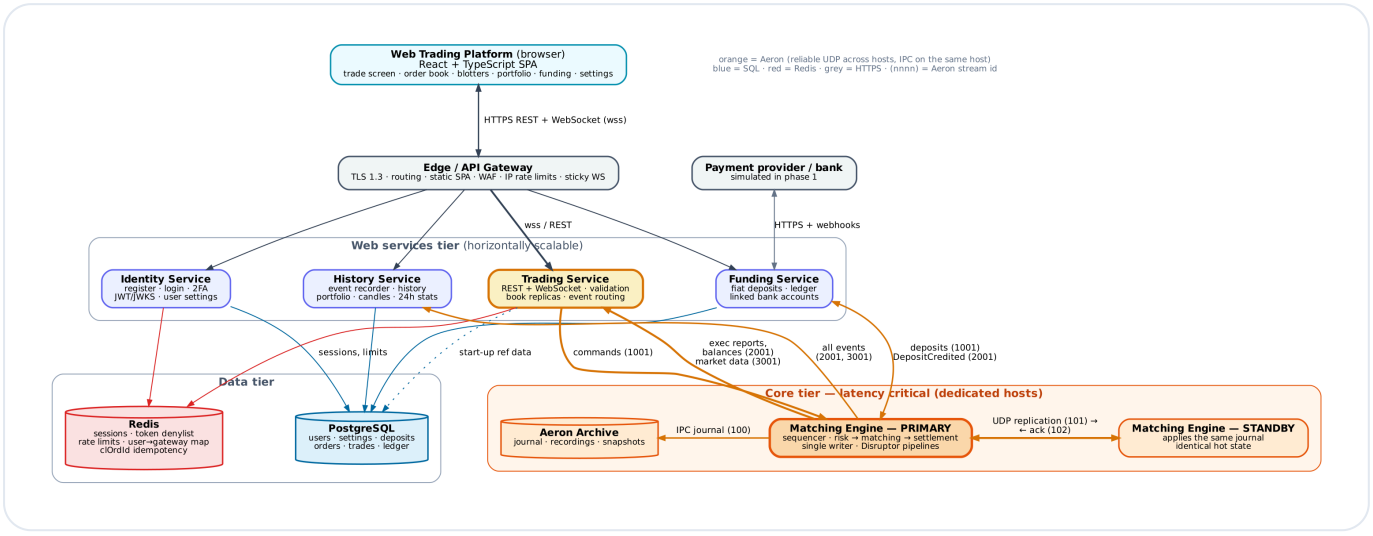
1. **Instruments** — fiat FX pairs only (EUR/USD, GBP/USD …) or crypto too? Affects decimals and custody.
2. **Fiat deposits** — real payment provider (Open Banking / Stripe / Modulr) or simulated bank for now? Design assumes a pluggable adapter, mock first.
3. **HA from day one?** — design supports a single-node engine first (Archive + snapshots), then primary/standby. Aeron Cluster (Raft) remains an option if automatic quorum is preferred over an arbiter.
4. **Hosting** — bare metal/VMs (best latency), or Kubernetes for non-latency-critical services only? Design assumes ME + Trading Service on tuned hosts, rest anywhere.
5. **Regulatory scope** — KYC level needed before trading/deposits.

1. Architecture

1.1 System context



1.2 Layered component view



The platform is organised in five tiers. Each tier talks only to the tier next to it or to the data tier. The one exception is the Aeron link between the web services tier and the core tier.

Tier	Components	Character
Client	Web Trading Platform (React SPA)	Untrusted; HTTPS + WSS only
Edge	Edge / API Gateway (Envoy or Nginx)	TLS termination, WAF, routing, IP rate limits, sticky WebSocket routing, serves the SPA bundle
Web services	Identity, History, Funding, Trading Service	Horizontally scalable; stateless apart from caches; Spring Boot (Identity, Funding, History REST) and Netty (Trading Service)
Core	Matching Engine PRIMARY, Matching Engine STANDBY, Aeron Archive	Latency critical; dedicated, tuned hosts; deterministic single-writer state machine

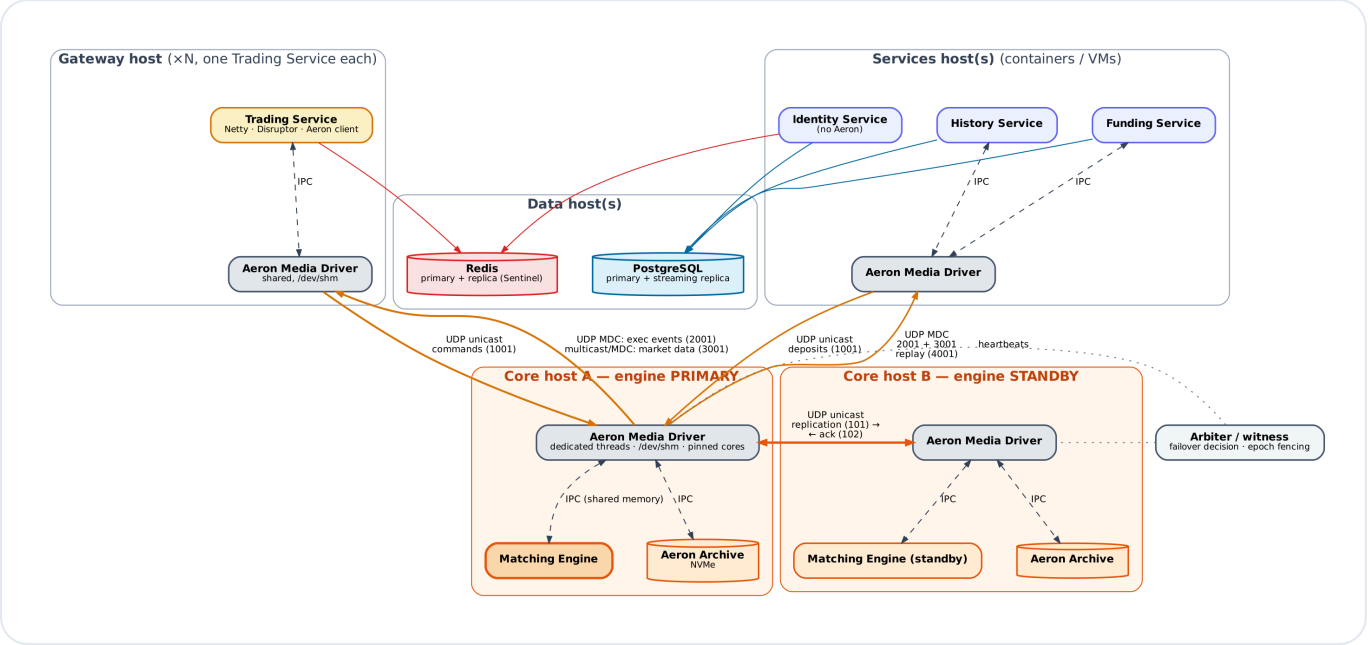
Tier	Components	Character
Data	PostgreSQL, Redis	Durable projections and shared caches. **Never on the matching hot path**

Colour legend used in all diagrams: **orange = Aeron** (IPC on the same host, reliable UDP across hosts), **blue = SQL**, **red = Redis**, **grey = HTTPS**. Numbers in brackets are **Aeron stream IDs** (see §3.1).

Component responsibilities

Component	Responsibilities	Owns state	Tech
Matching Engine (primary)	Sequences every command; journals; replicates to the standby; pre-trade risk (funds hold); price-time matching; settlement; exec reports; L2 market data; snapshots	Authoritative: order books, open orders, trading balances, sequence counters	Java 21, Aeron (driver, Archive), Disruptor 4, Agrona, SBE
Matching Engine (standby)	Consumes the replicated journal and applies it with the same code, so its state stays identical; acks; takes over on failover	Mirror of primary	same binary, `role=STANDBY`
Aeron Archive	Records the command journal (100) and the output streams; stores snapshots; serves replays (4001)	Journal and snapshots on NVMe	Aeron Archive
Trading Service	WS/REST for trading; JWT session auth; validation; idempotency; rate limiting; soft funds check; command encoding; event routing to users; L2 book replicas; conflated fan-out	Soft state only (rebuildable)	Java 21, Netty, Disruptor, Aeron client
Identity Service	Registration, email verification, login, 2FA, JWT/JWKS, refresh tokens, sessions/devices, user settings	PostgreSQL + Redis	Spring Boot 3, Spring Security
Funding Service	Fiat deposits through a PSP adapter (mock first), linked bank accounts, double-entry ledger, `Deposit` commands to the engine, reconciliation	PostgreSQL	Spring Boot 3, Aeron client
History Service	Event recorder: subscribes to 2001/3001 and fills gaps by Archive replay (4001); writes orders, fills, balances and candles to PostgreSQL; REST for history, portfolio, candles, 24h stats, CSV export	PostgreSQL (projection)	Java 21, Aeron client, JDBC batch/COPY, Spring Web
Edge / API Gateway	TLS 1.3, routing, WAF, IP rate limits, static SPA, sticky WS	none	Envoy / Nginx
PostgreSQL	Users, settings, ledger, deposits, order and trade history, candles	—	PostgreSQL 16 (+ streaming replica)
Redis	Shared, non-authoritative, fast state for the web services tier (see §1.5)	—	Redis 7 (primary + replica, Sentinel)

1.3 Deployment topology



Rules:

- **Same host → Aeron IPC.** Every host runs **one Aeron Media Driver**. Processes on the host exchange messages through its shared-memory log buffers in `/dev/shm`.
- **Across hosts → Aeron reliable UDP.** Use unicast for commands (1001) and replication (101/102). Use **MDC (multi-destination-cast) or multicast** for exec events (2001) and market data (3001), so one send from the engine reaches every gateway and service.
- **Core hosts** use the Media Driver in `DEDICATED` threading mode with busy-spin idle strategies. Hot threads are pinned to isolated cores (`isolcpus`), the Archive writes to local NVMe, and A and B have a direct, low-jitter network path (same rack and switch, 10/25 GbE).
- **Arbiter / witness**: a small process on a third host that watches heartbeats from both engines. It promotes the standby only when the primary is unreachable from both the witness and the standby. A monotonically increasing **epoch** fences off a stale primary, which prevents split-brain.
- **Gateway hosts**: one Trading Service per host, or a few with separate core pinning. Scale horizontally behind the Edge with sticky WebSocket sessions.
- **Services and data hosts** have no latency requirements and can run on VMs or Kubernetes.

1.4 Technology stack

Concern	Choice	Why
Language/runtime	Java 21 LTS; hot path allocation-free; ZGC (generational) elsewhere	Mature low-latency ecosystem
Messaging	Aeron (Media Driver, Archive); Aeron Cluster as an optional later upgrade (ADR-001)	Required; µs latency, reliable UDP, replay
Encoding	SBE (Simple Binary Encoding)	Zero-copy flyweights

Concern	Choice	Why
Inter-thread	LMAX Disruptor 4.x	Required; batching, single writer
Collections/buffers	Agrona (`Long2ObjectHashMap` , `UnsafeBuffer` , `IdleStrategy` , counters)	Primitive, allocation-free
Thread affinity	OpenHFT Java-Thread-Affinity	Pin hot threads
Gateway	Netty 4	Fine-grained control, low GC
Services	Spring Boot 3	Productivity
DB	PostgreSQL 16	Ledger integrity, rich queries
Cache / shared state	Redis 7 (Lettuce client)	Sessions, distributed limits, idempotency
Frontend	React 18, TypeScript, Vite, Tailwind + Radix/shadcn, Zustand, TanStack Query/Table, react-virtuoso, lightweight-charts	Modern, fast, accessible
Build	Gradle (Kotlin DSL) multi-module; pnpm	
Observability	Micrometer, Prometheus, Grafana, HdrHistogram, OpenTelemetry (non-hot paths), Loki	
Testing	JUnit 5, jqwik, JMH, Testcontainers, Playwright	

1.5 Redis usage (kept off the hot path)

Key space	Used by	Purpose	TTL
`sess:{sessionId}`	Identity, Trading Service	Session metadata, device, last seen	refresh-token lifetime
`deny:{jti}`	Identity, Trading Service	Revoked access/refresh tokens (logout, password change)	token expiry
`rl:{userId}:{bucket}`	Trading Service, Edge	Global per-user rate limits across instances, using Lua token bucket. Local buckets enforce in-process, and Redis is synced asynchronously every 100 ms	1–60 s
`clord:{userId}:{clOrdId}`	Trading Service	Cross-instance idempotency for client order IDs (SETNX). The engine remains the final duplicate check	24 h
`gw:{userId}`	Trading Service	Which gateway instances currently host a user's sessions (diagnostics and targeted kick-out)	session

Key space	Used by	Purpose	TTL
`ticker:{symbol}`	History Service	Cached 24h stats for REST	5 s
`login:fail:{email/ip}`	Identity	Brute-force protection	15 min

If Redis is unavailable, the services fall back gracefully: local rate limits only, a local idempotency cache, and the JWT signature check (the denylist check fails open only for access tokens younger than 60 s). Trading continues.

1.6 Architecture Decision Records (summary)

ADR	Decision	Alternatives considered	Rationale
001	**Primary/standby engines with a replicated journal** over Aeron (101/102), arbiter-based failover with epoch fencing. Aeron Cluster (3-node Raft) kept as an upgrade path	Aeron Cluster from day one; single node only	Lowest and most predictable latency (one RTT to the standby), simple to reason about, matches the desired topology. Raft adds automatic quorum at the cost of an extra node and some latency
002	**Trading balances and funds holds live in the Matching Engine**	Risk in Trading Service	Holds and fills serialised in one log: no double-spend, no distributed consistency problem
003	Fixed-point `long` price/qty, per-instrument scale	`BigDecimal`, `double`	No allocation, exact
004	Engine Disruptor pipeline: *journal ‖ replicate ‖ risk-hold → match → settle ‖ market data → release gate → publish*	Single thread	Parallel durability with processing; deterministic; one writer per shard
005	**Output release gate**: no event leaves the engine until its command is journaled locally *and* acked by the standby (sync mode)	Publish before durable	Clients never see a fill that could be lost on failover (RPO = 0)
006	Market orders = IOC with protection collar	Unbounded	Prevents sweeping thin books, bounds funds hold
007	Browser market data = L2 snapshot + sequenced, conflated deltas	Full book every tick	Bandwidth; still full depth with per-level liquidity
008	PostgreSQL is a **projection**; Redis is a **cache**; neither is authoritative for trading state	DB-first	Keeps storage off the hot path; rebuildable from the journal
009	Browser protocol JSON v1, binary later	Binary first	Debuggability

1.7 Proposed repository layout

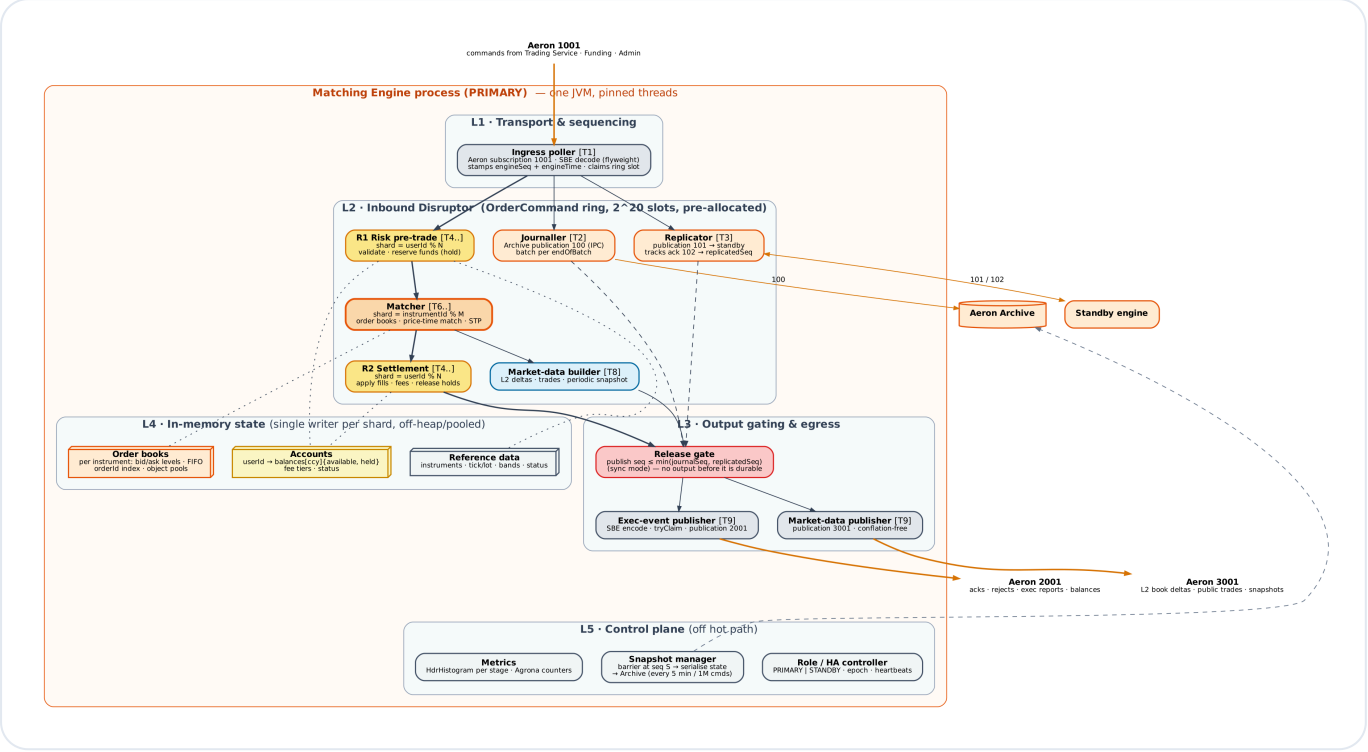
```
flash-exchange/
├─ docs/                          ← this design (+ diagrams/ sources)
├─ build-logic/                   ← Gradle conventions
├─ protocol/                      ← SBE XML schemas + generated codecs
├─ common/                        ← fixed-point math, ids, clocks, Aeron/Disruptor help
ers
├─ matching-engine/               ← sequencer, risk, order books, matching, HA, snapsho
ts
├─ matching-engine-bench/         ← JMH + latency harness
├─ arbiter/                       ← failover witness
├─ trading-service/               ← Netty WS/REST gateway, Aeron client, fan-out
├─ identity-service/
├─ funding-service/
├─ history-service/               ← event recorder + REST
├─ web/                           ← React SPA
├─ infra/                         ← docker-compose (dev), Ansible/Terraform, media-driv
er configs
└─ test-system/                   ← end-to-end & load tests (simulated traders)
```

2. Matching Engine

2.1 Goals

- Price-time (FIFO) priority matching for spot instruments.
- Limit orders (GTC default, IOC, FOK, post-only), market orders (IOC with price collar), cancel, cancel-all.
- **Predictable latency**: engine-internal p99 ≤ 10 µs and p99.9 ≤ 50 µs, excluding the standby round trip. No GC pauses on the hot path.
- Throughput: ≥ 1 M commands/s per engine across all instruments; ≥ 300 k/s on one hot instrument.
- **Deterministic and event-sourced**: the same journal always produces the same state and outputs. This is what makes hot standby, replay and audit possible.
- **RPO = 0** for acknowledged events (release gate, §2.8). Failover RTO < 1 s.

2.2 Layered internal architecture



Layer	Responsibility	Threads (default core map)
L1 Transport & sequencing	Poll Aeron subscription **1001**, decode SBE with flyweights, assign `engineSeq` (monotonic) and `engineTime` (the primary's clock, read once per command), copy into a pre-allocated ring slot	T1 ingress poller (core 2)
L2 Inbound Disruptor	`OrderCommand` ring, 2^20 slots. Parallel consumers: **Journaller** (Archive, stream 100), **Replicator** (stream 101 to the standby, consumes acks on 102), and the business chain **R1 risk → Matcher → R2 settlement**, plus the market-data builder	T2 journaller (core 3), T3 replicator (core 4), T4–T5 risk shards (cores 5–6), T6–T7 matcher shards (cores 7–8), T8 MD builder (core 9)

Layer	Responsibility	Threads (default core map)
L3 Output gating & egress	**Release gate**: hold results until the command is durable. Then SBE-encode into publications **2001** (private events) and **3001** (market data) using `tryClaim`	T9 publisher (core 10)
L4 In-memory state	Accounts (balances, holds, fee tier), order books, reference data. Each partition has exactly one writer thread	—
L5 Control plane	Snapshot manager, role/HA controller (epoch, heartbeats), admin commands, metrics (Agrona counters read by an exporter outside the hot path)	T10 (shared, non-isolated core)

The Media Driver on the same host uses three further isolated cores: conductor, sender and receiver.

Why R1 and R2 are separate stages on the same threads. Risk state is partitioned by `userId`, and matching is partitioned by `instrumentId`. One user's hold is placed by risk shard *k* before matching and released or settled by the same shard *k* after matching. Each account balance therefore has a single writer, with no locks and no CAS. This is the same structure as LMAX and exchange-core.

2.3 Command lifecycle (NewOrder, step by step)

#	Stage	Action
1	Ingress poller	Reads `NewOrder` from 1001. Rejects malformed or unsupported versions immediately with a `Reject` result. Claims ring sequence *s*, copies the fields, sets `engineSeq = s`, `engineTime = now`
2a	Journaller	Appends the raw command (with seq and time) to Archive publication 100. Advances `journalSeq` at end of batch
2b	Replicator	Sends the same bytes on 101. The standby acks the highest contiguous seq on 102. Sets `replicatedSeq`
2c	R1 risk (shard `userId % N`)	Checks user status, instrument status, static rules (tick, lot, min notional, price band) and duplicate `clientOrderId` (per-user open-order map). **Reserves funds**: BUY limit holds `price × qty × (1 + takerFee)` in quote; BUY market holds `quoteQty` or `collarPx × qty × (1 + fee)`; SELL holds `qty` of base. On failure, marks the slot `REJECTED(reason)` and later stages skip it
3	Matcher (shard `instrumentId % M`)	Matches against the book (§2.6). Appends `MatcherEvent`s (TRADE, CANCEL, ACCEPT) to the slot's pre-allocated event chain. Updates level aggregates
4a	R2 settlement (shard per involved user)	For every TRADE, moves balances for the taker and maker, charges fees and releases price-improvement or unused holds. Each shard handles only the users it owns, so a trade between users on different shards is settled in two halves by two shards. Both halves are deterministic
4b	MD builder	Turns level changes into `BookDelta` entries and `PublicTrade`s, keyed by `bookSeq`
5	Release gate	Waits until `s ≤ journalSeq` **and** (sync mode) `s ≤ replicatedSeq`

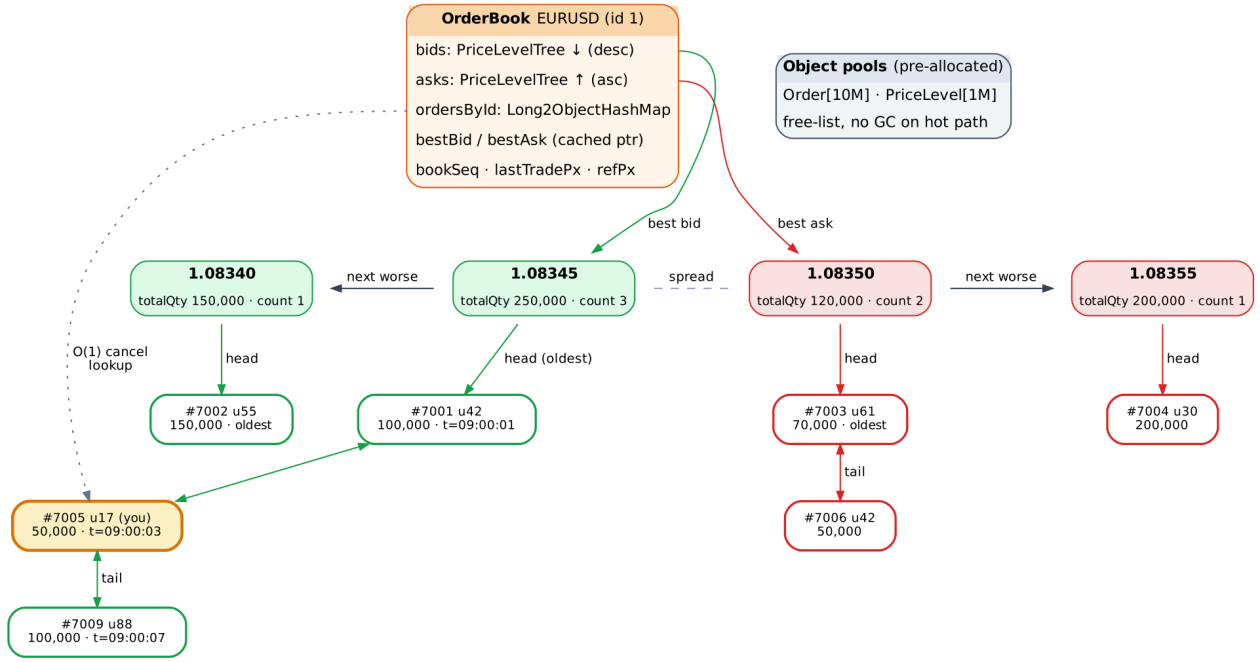
#	Stage	Action
6	Publisher	Encodes `OrderAccepted` / `ExecutionReport`s / `BalanceUpdate`s on 2001 and `BookDelta` / `PublicTrade` on 3001, then frees the slot

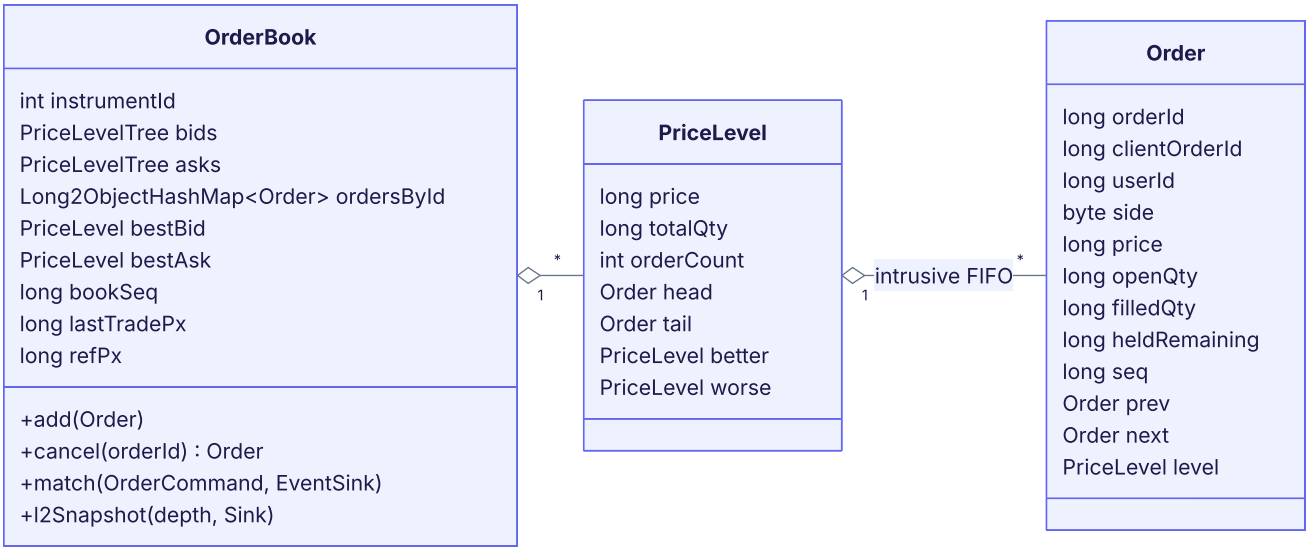
The gate does not stall the business stages. Matching for *s+1…s+k* continues while *s* waits for the ack. Only the publisher waits, so throughput is unaffected and latency rises by one standby round trip (≈ 15–30 µs on a local 25 GbE link).

2.4 Domain model

Concept	Representation
Instrument	`int instrumentId`; base/quote currency IDs; `priceScale`, `qtyScale`, `tickSize`, `lotSize`, `minQty`, `maxQty`, `minNotional`, price band %, collar %, status (`PRE_OPEN`, `TRADING`, `HALTED`, `CLOSED`)
Price / quantity	`long` fixed-point (EUR/USD price scale 5 → 1.08345 = `108345`; qty scale 2)
Notional	`price × qty` computed with `Math.multiplyHigh` overflow checks and rescaled to quote scale, rounding against the user for holds and exactly for fills
Order ID	`long`, engine-assigned from a counter in state (deterministic)
Client order ID	`long` from the client (gateway maps string → 64-bit hash plus a check), unique per user among open orders
Account	`userId` → `balances[currencyId] {available, held}`, `feeTier`, `status`, `openOrderCount`
Side / type / TIF	`BUY` / `SELL`; `LIMIT` / `MARKET`; `GTC` (default), `IOC`, `FOK`, `GTD` (reserved)
Flags	`POST_ONLY`; STP mode (`CANCEL_NEWEST` default, `CANCEL_OLDEST`, `CANCEL_BOTH`, `DECREMENT`)

2.5 Order book





Operation	Complexity	How
Best bid/ask	O(1)	cached pointers
Add at existing level	O(1)	hash lookup price → level, append to tail
Add new level	O(log L) tree / O(1) ladder	pooled node insert, link `better` / `worse`
Cancel	O(1)	`ordersById` → unlink from FIFO → adjust aggregates → remove empty level

Operation	Complexity	How
Match one fill	O(1)	head of best level
L2 snapshot depth *d*	O(d)	walk `worse` links

Implementation notes: price levels sit in a pooled red-black tree keyed by `long`, plus a `Long2ObjectHashMap<price, level>` for O(1) access. A dense **price ladder** (array indexed by `(price − base)/tick`) is optional for narrow-range FX instruments. `Order` and `PriceLevel` come from **pre-allocated pools** (sized from configuration, e.g. 10 M orders), are recycled on fill or cancel, and are never garbage-collected. Matching emits **at most K events per command** into the slot's fixed-size chain (K=64). A taker that would generate more is split internally into continuation slots, which keeps slot memory bounded.

2.6 Matching algorithm

```
match(cmd):
  book = books[cmd.instrumentId]
  opp  = cmd.side == BUY ? book.asks : book.bids
  limit = cmd.type == MARKET ? collar(book, cmd.side) : cmd.price
  if cmd.postOnly and crosses(opp.best, limit): reject(WOULD_TAKE); return
  if cmd.tif == FOK and availableQty(opp, limit) < cmd.qty: reject(FOK_NOT_FILLED); re
turn
  remaining = cmd.qty                          // or quote budget for market BUY-by-am
ount
  while remaining > 0 and opp.best != null and crosses(opp.best.price, limit):
      level = opp.best
      maker = level.head
      if maker.userId == cmd.userId: applySTP(cmd, maker); continue
      q = min(remaining, maker.openQty)        // budget-limited for quote-amount mark
et buys
      emitTrade(taker=cmd, maker, px=level.price, qty=q)   // maker price
      maker.openQty -= q; level.totalQty -= q; remaining -= q
      if maker.openQty == 0: unlink(maker); emitMakerFilled(maker); free(maker)
      if level.orderCount == 0: removeLevel(level)
  if remaining > 0:
      if cmd.type == LIMIT and cmd.tif == GTC: rest(cmd, remaining)  -> OrderAccepted
(NEW|PARTIALLY_FILLED)
      else: emitCancel(cmd, remaining, IOC_REMAINDER | MARKET_NO_LIQUIDITY)
```

Rules:

- **Price priority, then time priority.** Time is the engine sequence when the order rested, not a wall-clock time.
- **Execution price is the maker's price.** Price improvement goes to the taker, and the excess hold is released in R2.
- **Collar**: `bestOpposite × (1 ± collar%)`, default 5%, fixed at the start of matching.
- **Price band** (R1): reject limit orders outside ±band% of `refPx` (last trade, or mid if there is no trade).
- **STP** is checked per maker before filling.

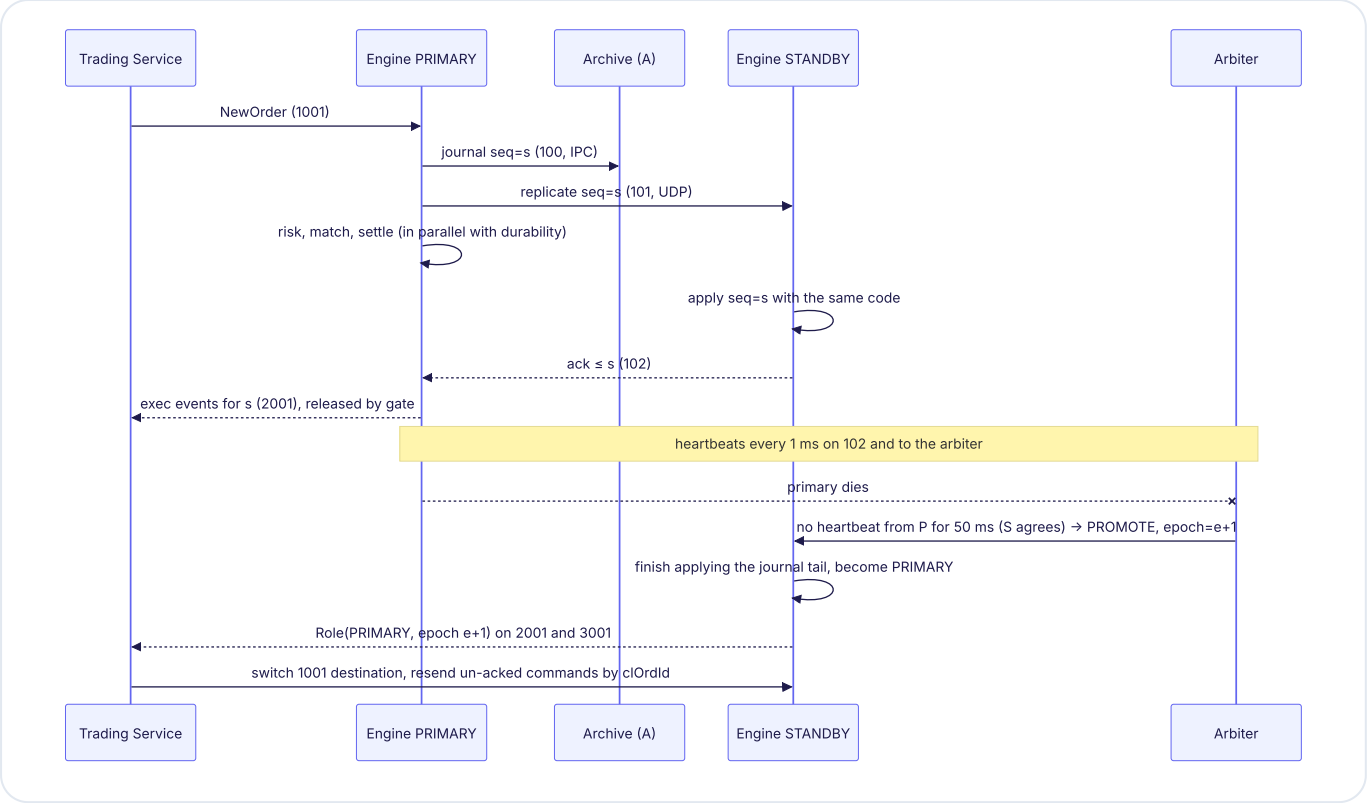
- A cancel that arrives after a fill gets `CancelRejected(TOO_LATE)`.

2.7 Risk & accounts

Event	Effect (single writer: risk shard owning the user)
`Deposit`	`available[ccy] += amt` (idempotent on `fundingTxId`, kept in a bounded dedup set and also in the snapshot)
New BUY limit	`available[quote] -= hold; held[quote] += hold`, where `hold = ceil(px × qty × (1 + takerFee))`
New SELL	`available[base] -= qty; held[base] += qty`
Fill (buyer)	`held[quote] -= px_exec × q × (1+fee_rate)`; `available[base] += q`; release price improvement `(px_limit − px_exec) × q × (1+takerFee)`
Fill (seller)	`held[base] -= q`; `available[quote] += px_exec × q × (1 − fee_rate)`
Fee	Credited to fee account `userId=0` in the received currency
Cancel / remainder / reject after hold	Release `order.heldRemaining`
`WithdrawRequest`	`available -= amt; pendingWithdrawal += amt` → `WithdrawResult` finalises or reverts

Invariant (asserted in tests and by a continuous checker on the standby): for each currency, Σ(available + held + pendingWithdrawal) over all accounts, plus the fee account, equals Σ deposits − Σ withdrawals.

2.8 Durability, replication and failover



- **Replication modes** (config): `SYNC` (default: gate waits for the standby ack, RPO 0) or `ASYNC` (gate waits only for the local journal, lower latency; possible loss of the last few ms on a host loss). Automatic downgrade from `SYNC` to `ASYNC` when the standby is lost, with an alert, so trading continues.
- **Determinism rules**: business logic never reads the clock, randomness or environment. Only `engineTime` from the command is used. Collections iterate in deterministic order. The JVM, schema and code versions are identical on both nodes, and the version is checked in the replication handshake.
- **Standby outputs**: the standby computes outputs but does not publish them. It may compare a hash of the outputs per batch with the primary's, via a periodic checksum on 102, to detect divergence early.
- **Epoch fencing**: every 2001/3001 message carries `epoch`. Consumers ignore lower epochs. A demoted primary that comes back sees the higher epoch and stays passive.
- **Re-joining**: a new standby loads the latest snapshot from the primary's Archive, replays the journal from the snapshot position (Archive replication), and then switches to live 101.

2.9 Snapshots and recovery

- Taken every 5 minutes or every 1 M commands, and on admin request. The snapshot manager publishes a `SnapshotBarrier` command through the ring. When every stage has passed it, the state is serialised (SBE, chunked) to an Archive recording, with `{snapshotSeq, journalPosition, epoch, checksum}`.
- Contents: reference data, accounts (balances, holds, fee tiers), books (orders in FIFO order per level), ID counters, deposit dedup set, `bookSeq` per instrument.
- **Cold recovery** (both nodes lost): load the latest snapshot, replay the journal from its position, verify the checksum, then open for trading. Target < 30 s for 10 M open orders.
- Snapshots are taken on the **standby** where possible, so the primary never pauses.

2.10 Commands and outputs

Inbound (1001): `NewOrder`, `CancelOrder`, `CancelAll`, `Deposit`, `WithdrawRequest`, `WithdrawResult`, `RequestUserState`, `RequestBookSnapshot`, `AddUser`, `SetUserStatus`, `AddInstrument`, `SetInstrumentStatus`, `SnapshotBarrier`, `Heartbeat`. Outbound: 2001 (`OrderAccepted`, `OrderRejected`, `ExecutionReport`, `OrderCancelled`, `CancelRejected`, `BalanceUpdate`, `DepositCredited`, `UserState`, `Role`), 3001 (`BookDelta`, `PublicTrade`, `BookSnapshot`, `InstrumentStatus`, `Heartbeat`). Details in §3.

2.11 Failure modes

Failure	Detection	Behaviour
Ring buffer full (burst)	`remainingCapacity`	Ingress poller stops polling, so Aeron back-pressure reaches the Trading Service, which returns `BUSY` to clients. Nothing is dropped silently
Publication back-pressured (slow consumer)	`tryClaim` returns `BACK_PRESSURED`	Publisher retries with bounded spin. MDC/multicast flow control drops the slowest receiver, which resyncs via snapshot
Standby lost	No ack or heartbeat for 50 ms	Downgrade to `ASYNC`, alert, re-join when back
Primary lost	Arbiter and standby agree	Promote standby, epoch + 1 (§2.8)
Archive disk full	Archive error and disk metric alerts at 70%	Halt new orders (instrument status `HALTED`); cancels still allowed
Arithmetic overflow / invariant breach	Checked math, assertions	Reject the command (`INTERNAL`), alert. Never corrupt state

2.12 Performance engineering checklist

- `-Xms = -Xmx`, `-XX:+AlwaysPreTouch`, large pages, ZGC generational. Epsilon GC for benchmarks to prove zero allocation.
- `isolcpus`, `nohz_full`, `rcu_nocbs` on hot cores; IRQ affinity away from them; `performance` governor; deep C-states off; no SMT siblings on hot cores.
- Aeron: `ThreadingMode.DEDICATED`, `BusySpinIdleStrategy`, term buffers in `/dev/shm`, socket buffers ≥ 2 MB, MTU 1408 (or jumbo 8 KB on the core link).
- Warm-up: replay a synthetic order flow against a shadow instrument on start-up before going live (JIT).
- Latency measured with HdrHistogram per stage. The ingress timestamp is carried in the slot. Exported every second.
- Sizing: 1 M orders/s × ~128 B journal ≈ 128 MB/s. NVMe and the 25 GbE replication link have ample headroom.

3. Messaging & Protocol

3.1 Aeron stream and channel plan

Stream	Name	Producer → Consumers	Same host	Across hosts
1001	Commands	Trading Service, Funding, Admin → Engine primary	`aeron:ipc`	`aeron:udp?endpoint=core-a:40101` (unicast; after failover the clients switch to `core-b`)
2001	Exec events (private)	Engine → Trading Services, Funding, History	`aeron:ipc`	`aeron:udp?control=core-a:40201|control-mode=dynamic` (**MDC**: each subscriber registers, the engine sends once)
3001	Market data	Engine → Trading Services, History	`aeron:ipc`	UDP **multicast** `aeron:udp?endpoint=239.10.1.1:40301|interface=…` or MDC where multicast is unavailable (cloud)
100	Command journal	Engine → local Archive (recorded)	`aeron:ipc`	—
101	Replication	Primary → Standby	—	`aeron:udp?endpoint=core-b:40111` (unicast, dedicated NIC/VLAN)
102	Replication ack + heartbeat + checksum	Standby → Primary	—	`aeron:udp?endpoint=core-a:40112`
4001	Archive replay	Archive → History / recovering Trading Service / re-joining standby	`aeron:ipc`	`aeron:udp?endpoint=<consumer>:40401` (per replay session)
9001	Arbiter heartbeats / role commands	Arbiter ↔ engines	—	unicast UDP

Every message on 2001 and 3001 carries `engineSeq` and `epoch`. Every message on 3001 also carries the per-instrument `bookSeq`.

Notes:

- **Reliable UDP**: Aeron NAK-based retransmission. For MDC and multicast, use `min` flow control with tagged receivers so the engine is never slowed by one slow subscriber. A lagging receiver is dropped, detects the gap and resyncs (snapshot for 3001, replay 4001 or `RequestUserState` for 2001).
- **Term lengths**: 64 MB on 3001; 16 MB on 1001, 2001 and 101. MTU 1408 (jumbo frames optional on the core link).
- **Publication API**: `tryClaim` for messages under MTU (zero copy), `offer` otherwise. Back-pressure is absorbed only by the publisher stage, never by the matcher.

3.2 Message envelope (SBE)

Every message: SBE `messageHeader` (blockLength, templateId, schemaId, version) + common fields:

Field	Type	Notes
`correlationId`	int64	set by sender, echoed in responses
`ingressTimestampNs`	int64	latency tracing
`engineSeq`	int64	outbound only; monotonic, gap detection & idempotent replay
`engineTime`	int64	outbound only; primary clock captured at sequencing (journaled → deterministic)
`epoch`	int32	outbound only; HA fencing epoch

Schema versioning: additive fields only within a major schema version; `sinceVersion` attributes; consumers ignore unknown templates.

3.3 Inbound commands (stream 1001 → Matching Engine)

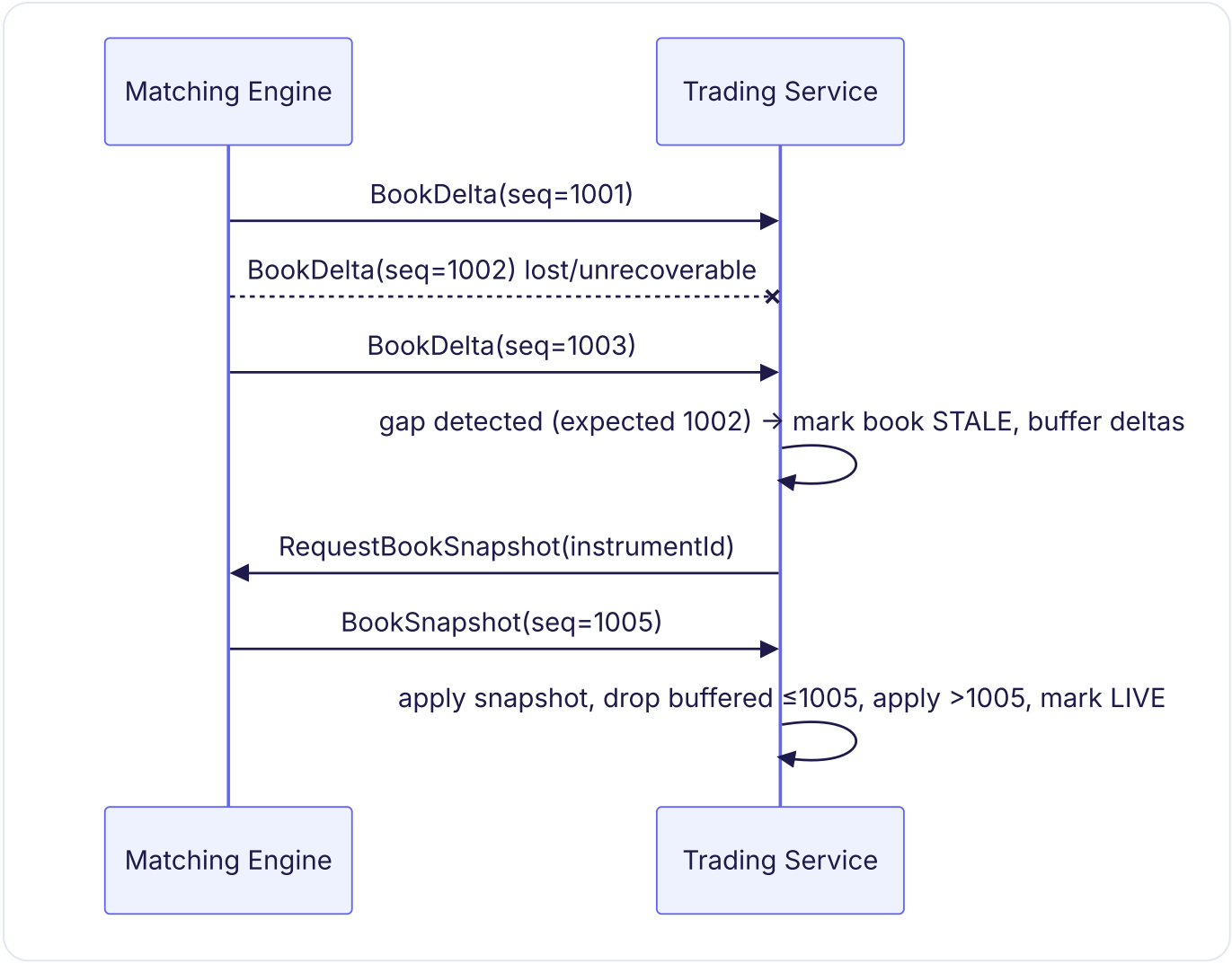
Template (templateId)	Key fields
`NewOrder` (1)	userId, clientOrderId, instrumentId, side, orderType, timeInForce, price, quantity, quoteQuantity (market buy by amount), flags (postOnly, stpMode), expireTime (GTD, reserved)
`CancelOrder` (2)	userId, instrumentId, orderId | clientOrderId
`CancelAll` (3)	userId, instrumentId (0 = all)
`Deposit` (10)	userId, currencyId, amount, fundingTxId
`WithdrawRequest` (11) / `WithdrawResult` (12)	userId, currencyId, amount, fundingTxId, outcome
`RequestUserState` (20)	userId
`RequestBookSnapshot` (21)	instrumentId, depth

Template (templateId)	Key fields
Admin: `AddInstrument` (30), `SetInstrumentStatus` (31), `AddUser` (32), `SetUserStatus` (33)	—

3.4 Outbound events (Matching Engine → 2001 / 3001)

Template (templateId)	Stream	Key fields
`OrderAccepted` (101)	2001	orderId, clientOrderId, userId, instrumentId, side, type, tif, price, qty, status
`OrderRejected` (102)	2001	clientOrderId, rejectReason (enum: INSUFFICIENT_FUNDS, INVALID_PRICE, INVALID_QTY, BELOW_MIN_NOTIONAL, INSTRUMENT_HALTED, DUPLICATE_CLORDID, WOULD_TAKE, FOK_NOT_FILLED, OUTSIDE_PRICE_BAND, USER_SUSPENDED, UNKNOWN_ORDER)
`ExecutionReport` (103)	2001 (to both maker & taker)	orderId, tradeId, userId, side, liquidity (MAKER/TAKER), lastPrice, lastQty, cumQty, leavesQty, avgPrice, fee, feeCcy, orderStatus
`OrderCancelled` (104)	2001	orderId, reason (USER, IOC_REMAINDER, STP, MARKET_NO_LIQUIDITY, ADMIN), leavesQty
`CancelRejected` (105)	2001	orderId, reason
`BalanceUpdate` (110)	2001	userId, currencyId, available, held, reason, refId
`UserState` (120)	2001	repeating groups: balances, open orders
`BookDelta` (201)	3001	instrumentId, bookSequence, repeating group {side, price, totalQty (0 = remove level), orderCount}
`PublicTrade` (202)	3001	instrumentId, tradeId, price, qty, aggressorSide, time
`BookSnapshot` (203)	3001	instrumentId, bookSequence, depth, bids[], asks[]
`InstrumentStatus` (204)	3001	instrumentId, status
`DepositCredited` (111)	2001	userId, currencyId, amount, fundingTxId
`Role` (130)	2001, 3001	role, epoch, primary endpoint
`Heartbeat` (250)	2001, 3001	engineSeq (lets consumers detect silence/gaps)

3.5 Sequencing and gap recovery



- Aeron reliable UDP already retransmits; application-level gaps only occur after a slow-consumer disconnect or restart.
- Private-event (2001) gaps are recovered by Archive replay (4001) from the last `engineSeq`, or by `RequestUserState`.

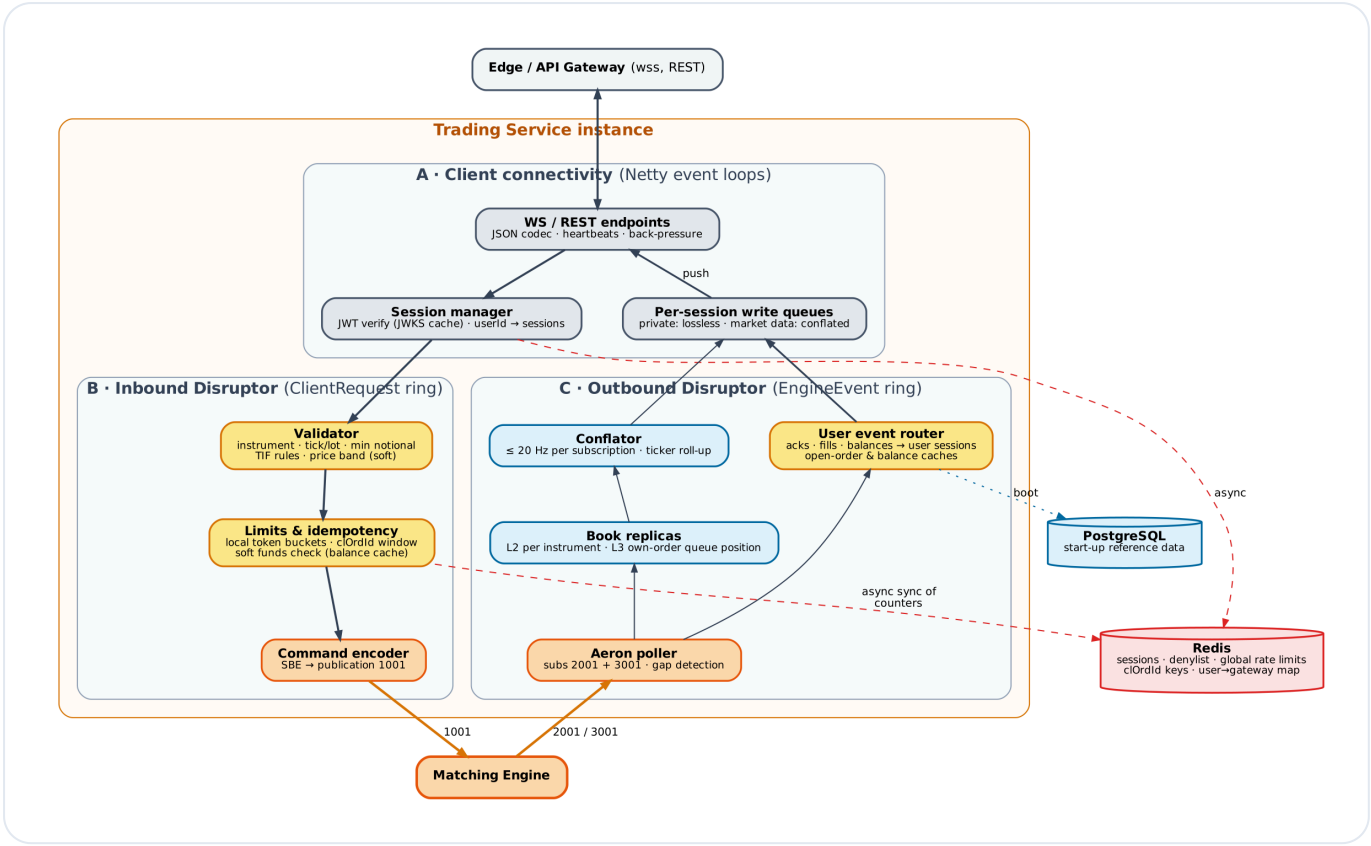
3.6 Browser protocol (Trading Service ↔ Web) — summary

Detailed in Trading Service §4.5. JSON over WSS v1; decimals sent as **strings** to avoid float rounding.

4. Trading Service

The bridge between the web platform and the Matching Engine.

4.1 Internal design



Layer	Components	Threads
A · Client connectivity	WS/REST endpoints (JSON codec, heartbeats, back-pressure), session manager (JWT verify with cached JWKS, `userId → sessions`), per-session write queues	Netty boss + worker event loops
B · Inbound Disruptor	Validator → limits & idempotency (local token buckets, clOrdId window, soft funds check) → command encoder (SBE → 1001)	1 handler chain, pinned
C · Outbound Disruptor	Aeron poller (2001 + 3001, gap and epoch checks) → user event router (acks, fills, balances; open-order and balance caches) ‖ book replicas (L2, own-order queue position) → conflator	1 poller + 1 chain, pinned
Side stores	Redis (sessions, denylist, global rate limits, clOrdId keys, user→gateway map), accessed **asynchronously** and never awaited on the order path; PostgreSQL read once at boot for reference data	—

Threads: Netty boss/worker group (I/O), 1 inbound Disruptor handler chain, 1 Aeron poller (busy-spin/backoff), 1 outbound chain (router + books), conflator timer on the event loop. Per-session write queues are bounded; a client that

cannot keep up gets dropped market-data frames (conflation), never dropped private events — if the private queue overflows the session is closed and must resync.

4.2 Validation (before the engine)

Check	Reject code
JWT valid, user active, KYC tier allows trading	`UNAUTHORIZED` / `FORBIDDEN`
Instrument exists & `TRADING`	`INSTRUMENT_UNAVAILABLE`
Price multiple of tick, qty multiple of lot, min/max qty, min notional	`INVALID_PRICE` / `INVALID_QTY` / `BELOW_MIN_NOTIONAL`
Type/TIF combo legal (MARKET ⇒ IOC; GTC default if absent)	`INVALID_TIF`
Price within band of last reference price (soft)	`OUTSIDE_PRICE_BAND`
`clientOrderId` not seen in last 24h for user (local LRU + Redis `SETNX clord:{user}:{id}`)	`DUPLICATE_CLORDID` (returns original status)
Rate limit (e.g. 50 orders/s, 500 open orders per user/instrument; local bucket, globally reconciled via Redis)	`RATE_LIMITED`
Soft funds check using cached balances	`INSUFFICIENT_FUNDS` (engine re-checks authoritatively)

Static validation is repeated cheaply in the engine for defence in depth, but the engine's authoritative checks are funds, duplicate orderId and instrument status.

4.3 Routing of engine events

- Every Trading Service instance publishes commands on **1001**, stamped with its `gatewayId`, and subscribes to **2001** (MDC). The engine sends each private event once, and every gateway receives it.
- The user event router keeps only events for users with a session on this instance. This also covers maker fills for users connected elsewhere. At larger scale, 2001 can be partitioned into N streams by `userId % N` and users pinned to gateway groups by consistent hashing at the Edge.
- Router keeps `userId → Set<Session>` (multiple tabs/devices) and pushes `order`, `fill`, `balance` messages to all of them.

4.4 Market-data fan-out

- Maintains a full L2 replica per instrument from `BookSnapshot` + `BookDelta`.
- On client subscribe: send snapshot (top N levels, default 50, or full) with `seq`, then deltas.
- Conflation: per subscription, coalesce changed levels within a 50 ms window and send one `book.delta` with the latest totals per level → bounded bandwidth, browser never sees an inconsistent level.

- **Own-order highlight**: client marks levels where it has open orders (from the private order feed). Optionally the server adds `queueAhead` per own order (sum of qty ahead in FIFO), derived from an L3 view the engine provides only to the Trading Service (not exposed publicly).
- Public trades and 24h ticker (open/high/low/last/volume, rolled by the service) streamed on `trades.<sym>` and `ticker.<sym>`.

4.5 WebSocket API

Endpoint `wss://api.flash.exchange/ws` — auth via short-lived JWT in first `auth` message (not in URL).

Client → server

```
{"op":"auth","token":"<jwt>"}
{"op":"subscribe","channels":["book.EURUSD:50","trades.EURUSD","ticker.*","orders","fi
lls","balances"]}
{"op":"unsubscribe","channels":["book.EURUSD"]}
{"op":"order.new","reqId":17,"clOrdId":"c-8f2a","symbol":"EURUSD","side":"BUY","typ
e":"LIMIT","tif":"GTC","price":"1.08345","qty":"10000.00","postOnly":false}
{"op":"order.new","reqId":18,"clOrdId":"c-8f2b","symbol":"EURUSD","side":"BUY","typ
e":"MARKET","quoteQty":"5000.00"}
{"op":"order.cancel","reqId":19,"orderId":"700013"}
{"op":"order.cancelAll","reqId":20,"symbol":"EURUSD"}
{"op":"ping"}
```

Server → client

```
{"ch":"book.EURUSD","type":"snapshot","seq":88112,"bids":[["1.08340","250000.00",
4]],"asks":[["1.08350","120000.00",2]]}
{"ch":"book.EURUSD","type":"delta","seq":88113,"bids":[["1.08340","150000.00",3]],"ask
s":[]}
{"ch":"trades.EURUSD","data":[{"id":"55021","px":"1.08350","qty":"5000.00","side":"BU
Y","ts":1793750400123}]}
{"ch":"orders","data":{"orderId":"700013","clOrdId":"c-8f2a","status":"NEW","px":"1.08
345","qty":"10000.00","filled":"0","avgPx":null,"tif":"GTC","ts":...}}
{"ch":"fills","data":{"tradeId":"55022","orderId":"700013","px":"1.08345","qty":"4000.
00","liq":"MAKER","fee":"0.43","feeCcy":"USD"}}
{"ch":"balances","data":[{"ccy":"USD","available":"9567.20","held":"6500.18"}]}
{"op":"ack","reqId":17,"ok":true}
{"op":"ack","reqId":18,"ok":false,"code":"INSUFFICIENT_FUNDS","msg":"..."}
```

Delta semantics: `[price, totalQty, orderCount]`; `totalQty == "0"` removes the level. A client seeing `seq` ≠ last+1 re-subscribes.

REST (same service, for clients that prefer it): `POST /api/v1/orders`, `DELETE /api/v1/orders/{id}`, `DELETE /api/v1/orders?symbol=`, `GET /api/v1/orders/open`, `GET /api/v1/balances`. History endpoints live in the History Service.

4.6 Resilience

- On start: load reference data from PostgreSQL, subscribe to 2001/3001, request book snapshots, and send `RequestUserState` for each user as they connect. Then go live.
- Engine failover: on `Role(PRIMARY, epoch+1)` the encoder switches the 1001 destination to the new primary. Commands with no ack for 250 ms are resent with the same `clientOrderId`, and the engine deduplicates them. Events with an old epoch are discarded.
- Redis outage: falls back to local rate limits and a local idempotency cache, and trading continues (see §1.5).
- Optional **cancel-on-disconnect** flag per session (off by default for web users).

5. Web Trading Platform

5.1 Feature list

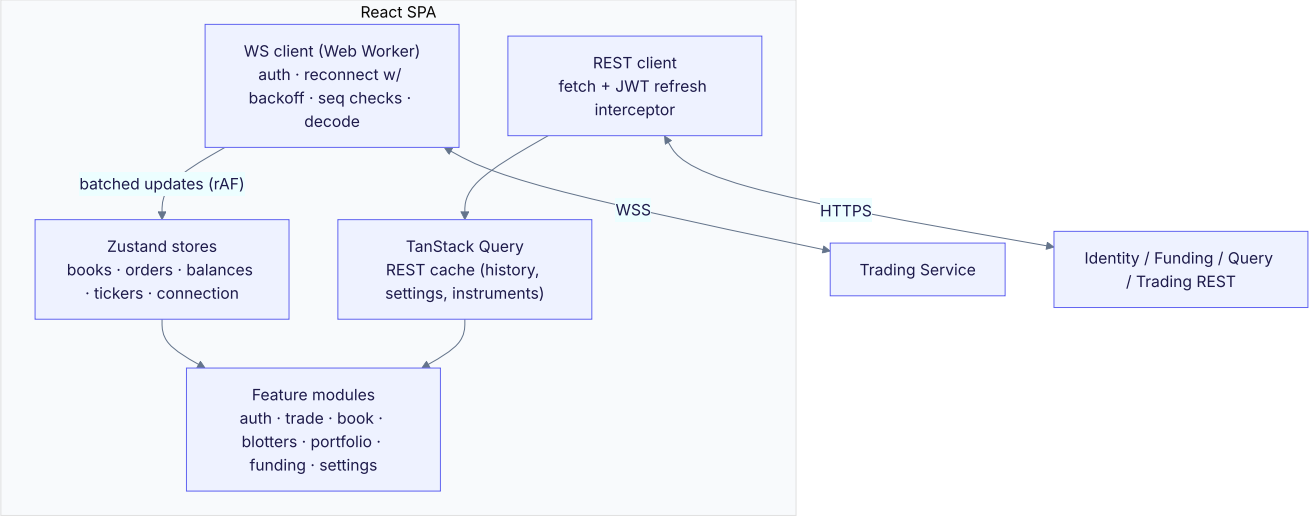
Area	Features
Onboarding	Register (email, password, country, T&C), email verification, login, 2FA (TOTP) enrolment, forgot/reset password, logout, active sessions/devices list
Settings	Theme: Light / Dark / System; colour scheme (accent palette; "green-up/red-down" or "red-up/green-down" for regional convention; colour-blind safe palette); default instrument; default order type & TIF (GTC preselected); order confirmation dialog on/off; number/date format; notification preferences; 2FA; change password
Funding	Deposit fiat (choose currency, amount, method → PSP redirect / bank transfer reference), deposit history with status, (phase 6) withdrawals to verified bank account
Market	Instrument selector with search & favourites; 24h ticker (last, change %, high, low, volume); price chart (candles 1m–1D, volume)
Order entry	Buy/Sell tabs; Limit / Market; TIF dropdown (GTC preselected, IOC, FOK); price, quantity, total (two-way calculated); % of available buttons (25/50/75/100); post-only checkbox; estimated fee; available balance; client-side validation (tick/lot/min notional); click on a book level fills price; keyboard shortcuts; confirmation dialog (optional)
Order book	Bid and ask side by side (or stacked on mobile), price-time priority order, per-level total quantity, order count, cumulative depth bar; spread & mid; precision grouping (e.g. 0.00001 / 0.0001 / 0.001); flash on change; **user's own levels highlighted** with marker + own qty and (optional) queue position tooltip
Recent trades	Public tape with aggressor colouring
Open orders blotter	Live: time, symbol, side, type, TIF, price, qty, filled, remaining, avg price, status (NEW / PARTIALLY_FILLED), cancel button, cancel-all; progress bar for partial fills
Order history	Filled / cancelled / rejected orders, filterable, CSV export
Trades blotter	User's fills: time, symbol, side, price, qty, notional, fee, maker/taker; filters, CSV export
Portfolio	All assets: available, in orders (held), total, valuation in reporting currency (USD/EUR/GBP selectable), allocation chart, P&L (realised, phase 6)
Notifications	Toasts for ack/reject/fill/cancel; optional sound; connection status indicator (live / reconnecting / stale)

5.2 Layout (desktop)

```
┌──────────────────────────────────────────────────────────────────────────┐
| Logo | EUR/USD ▾ 1.08345 +0.12% H 1.0851 L 1.0812 Vol 1.2B | Deposit | ⚙ | 👤|
├──────────────────────────────┬──────────────────────────┬────────────────┤
|                              | ORDER BOOK               | ORDER ENTRY    |
|        PRICE CHART           |  Bids      |   Asks      | [Buy] [Sell]   |
|                              | qty   px   | px    qty   | Limit | Market |
|                              | ...   ▶own | ...         | Price  Qty  Total |
|                              | spread 0.00010           | TIF [GTC ▾]    |
├──────────────────────────────┴──────────────────────────┤ [ Place order ]  |
| Open orders | Order history | Trades | Portfolio        | Balances       |
| (virtualised live blotters)                           | Recent trades  |
└──────────────────────────────────────────────────────────────────────────┘
```

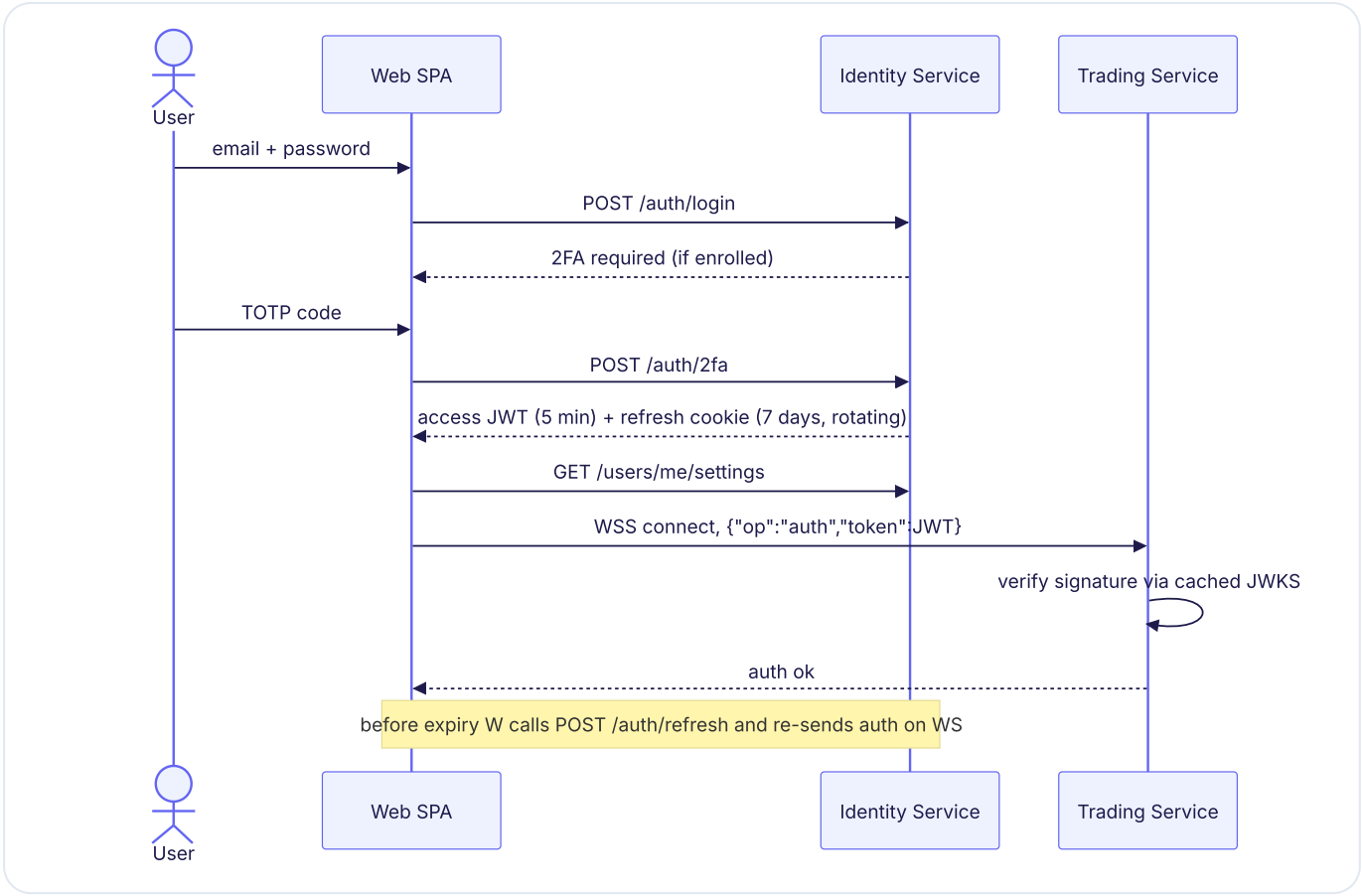
Responsive: ≥1280 px full grid (panels resizable, layout persisted in settings); 768–1279 px two columns; < 768 px tabbed (Chart / Book / Trade / Orders) with bottom sheet order entry.

5.3 Front-end architecture



- WebSocket decoding and book maintenance run in a **Web Worker**; UI receives batched state at `requestAnimationFrame` cadence → smooth at high update rates.
- Decimal arithmetic via `decimal.js` / `big.js` (never JS floats for money).
- Blotters virtualised (react-virtuoso / TanStack Table).
- Theming via CSS custom properties + Tailwind `dark:`; settings loaded at login and cached in `localStorage` to avoid flash of wrong theme.
- Accessibility: WCAG 2.1 AA, keyboard navigation, colour-blind palette.
- Security: access token in memory, refresh token in `HttpOnly; Secure; SameSite=Strict` cookie, CSP, no third-party scripts on trading pages.

5.4 Auth flow


User
Web SPA
Identity Service
Trading Service
email + password
POST /auth/login
2FA required (if enrolled)
TOTP code
POST /auth/2fa
access JWT (5 min) + refresh cookie (7 days, rotating)
GET /users/me/settings
WSS connect, {"op":"auth","token":JWT}
verify signature via cached JWKS
auth ok
before expiry W calls POST /auth/refresh and re-sends auth on WS
Web SPA
Identity Service
Trading Service
User

6. Accounts, Funding & Data

6.1 Balance ownership

Balance view	Owner	Purpose
Trading balance (`available` , `held`)	**Matching Engine** (authoritative)	Funds holds and settlement on the hot path
Ledger (double-entry journal)	**Funding Service** / PostgreSQL	Money in/out of the platform, regulatory record
Balance history / portfolio	**History Service** / PostgreSQL	Projection of engine `BalanceUpdate` events for UI

A nightly + continuous **reconciliation job** checks: ledger customer liability per currency == Σ engine balances (from latest snapshot / projection). Breaks raise alerts and can freeze affected accounts.

6.2 Deposit flow (summary — full sequence in 07)

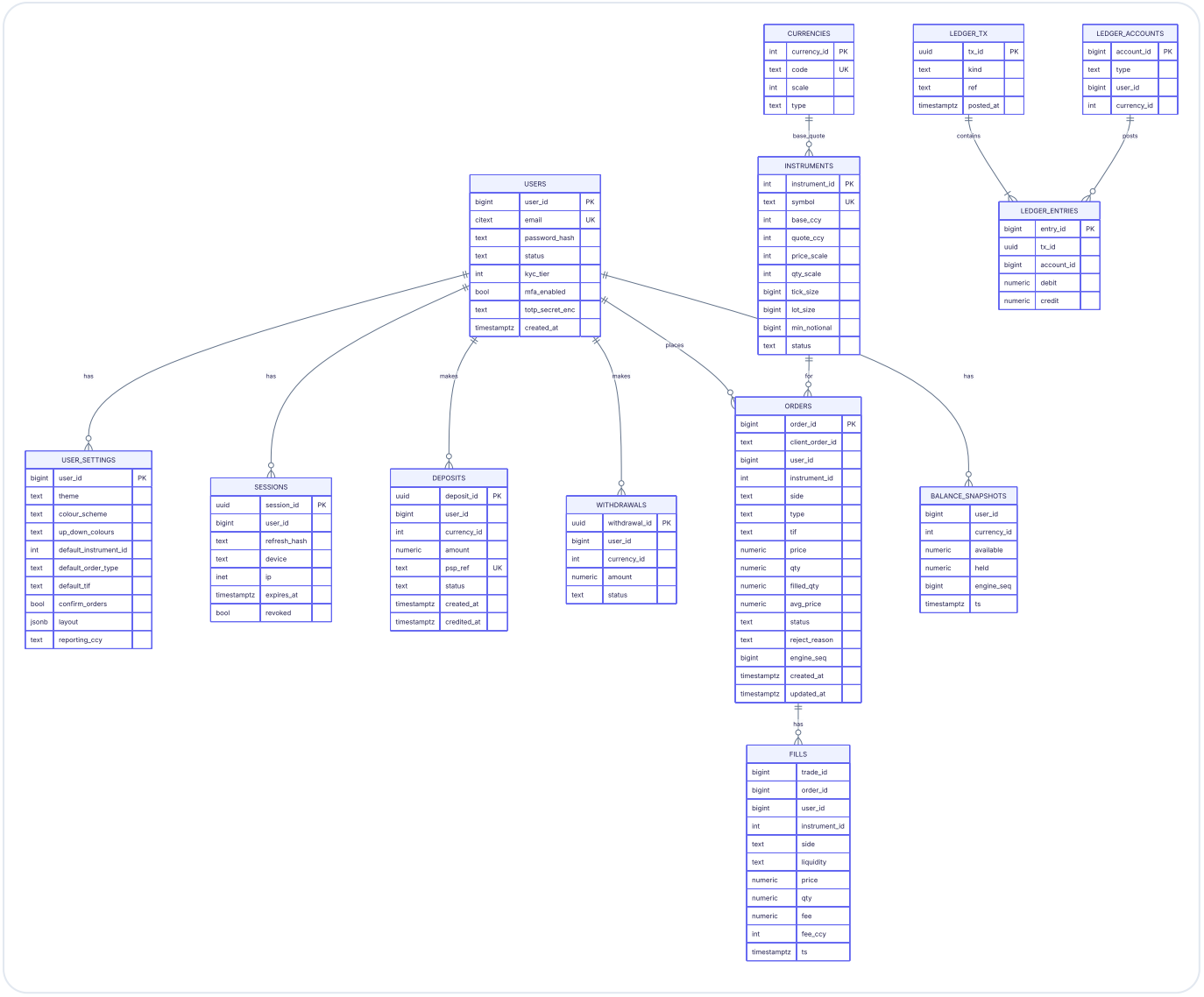
1. User initiates deposit → Funding creates `deposit(PENDING)` and PSP payment intent.
2. PSP confirms (webhook, signature verified, idempotent on PSP reference).
3. Funding posts ledger entries `DR bank_clearing / CR customer_liability:<user>` and status `CONFIRMED` .
4. Funding sends `Deposit` command (fundingTxId = idempotency key) to the Matching Engine.
5. Engine credits `available` , emits `BalanceUpdate` → Funding marks `CREDITED` ; Trading Service pushes balance to the user.

Phase 1 ships a **Mock PSP** (simulated bank with instant/delayed/failed outcomes) behind a `PaymentProvider` interface.

6.3 Fees

Maker/taker fee schedule per instrument and user tier (default maker 0.05%, taker 0.10%, configurable). Fees charged in the currency received, accrued to a platform fee account inside the engine, reported in `ExecutionReport` .

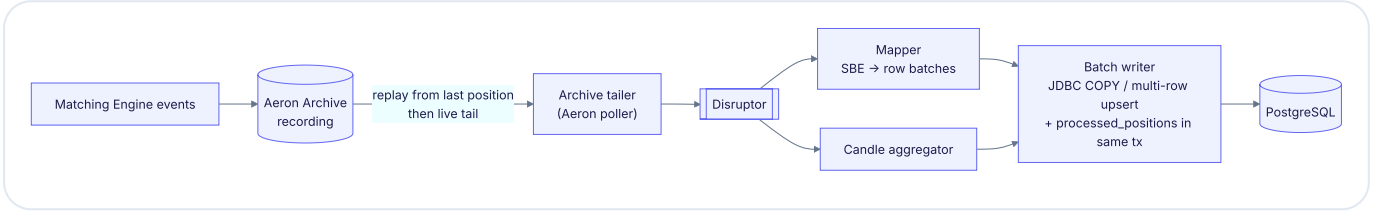
6.4 PostgreSQL schema (main tables)



Additional: `candles(instrument_id, interval, open_time, o,h,l,c, volume)` (TimescaleDB hypertable optional), `audit_log`, `processed_positions(consumer, archive_position, engine_seq)` for exactly-once projection, `api_keys` (phase 6).

Indexes: `orders(user_id, status, created_at desc)`, `fills(user_id, ts desc)`, `fills(instrument_id, ts)`. Partition `orders` / `fills` monthly.

6.5 Persistence pipeline



Exactly-once effect: rows and the consumer's `engine_seq` are committed in one transaction; on restart replay resumes from the stored archive position and skips `engine_seq ≤ stored`.

6.6 History Service REST

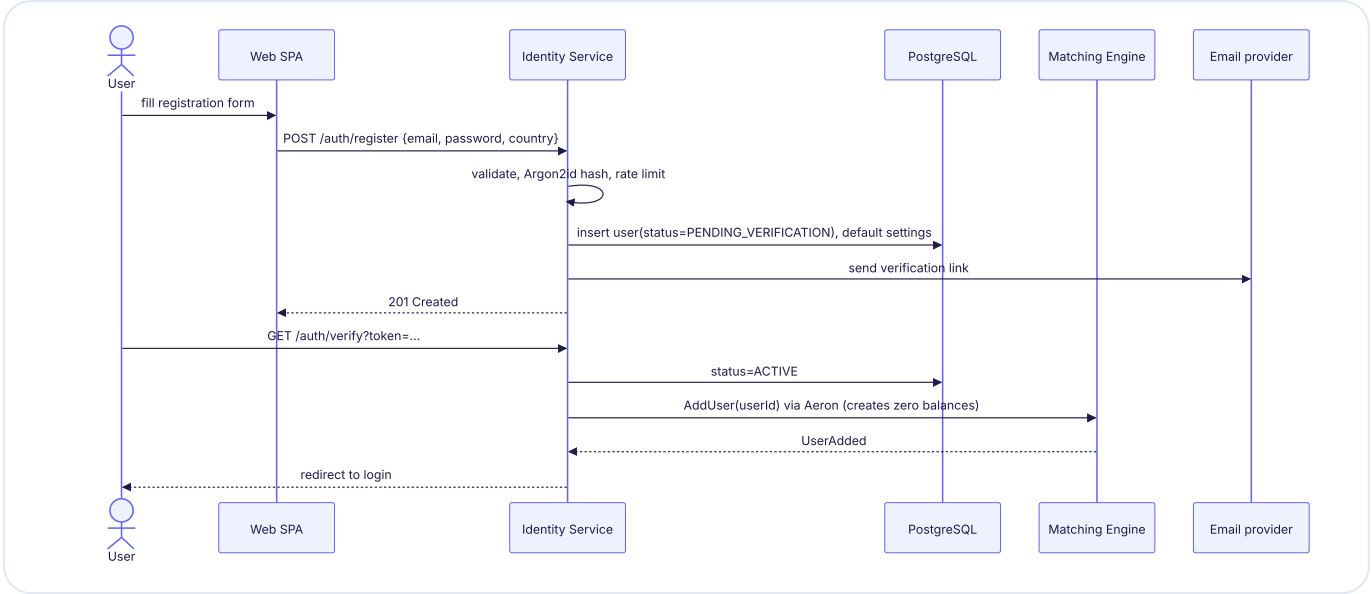
`GET /api/v1/orders?status=&symbol=&from=&to=&cursor=` · `GET /api/v1/fills?...` · `GET /api/v1/portfolio` (balances + valuation from latest mid prices) · `GET /api/v1/candles?symbol=&interval=&from=&to=` · `GET /api/v1/instruments` · `GET /api/v1/export/{orders|fills}.csv`. Cursor-based pagination.

6.7 Redis data model

See §1.5 for the key spaces. Deployment: Redis 7 primary + replica with Sentinel (3 sentinels), AOF `everysec`. All keys have TTLs, and **no key holds data that can't be rebuilt**. Losing Redis degrades rate limiting and idempotency to per-instance and forces users to log in again, but never affects balances, orders or trades.

7. Interactions

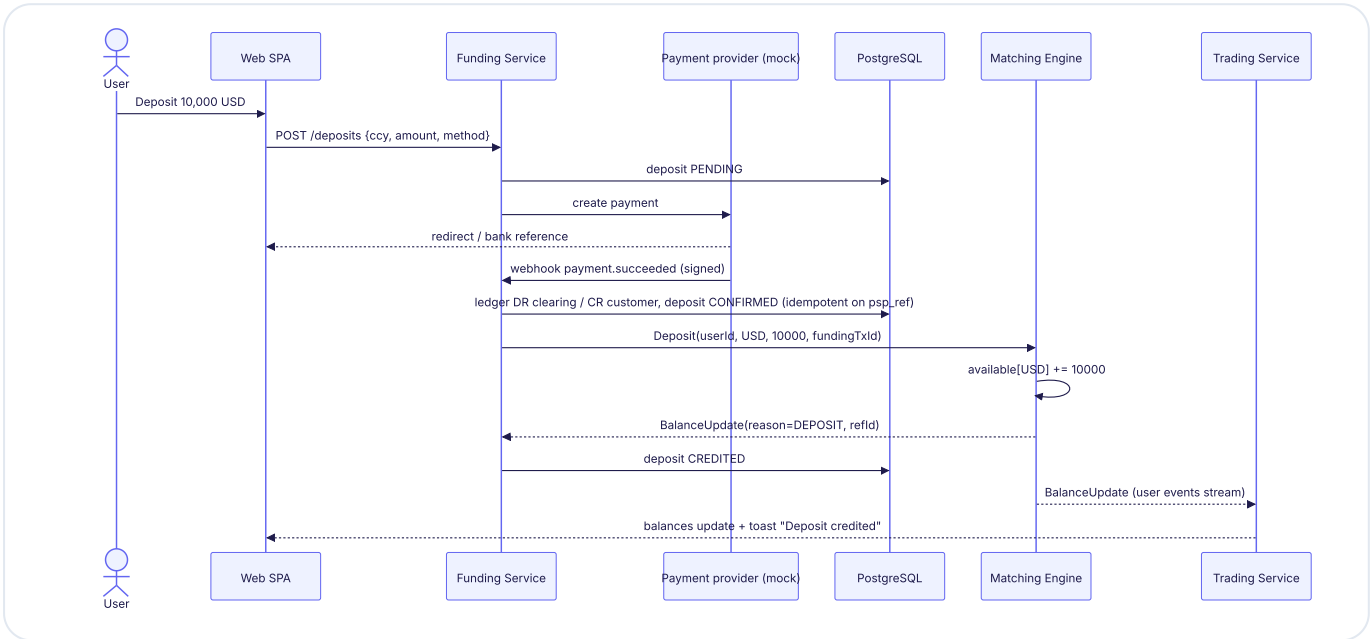
7.1 Registration



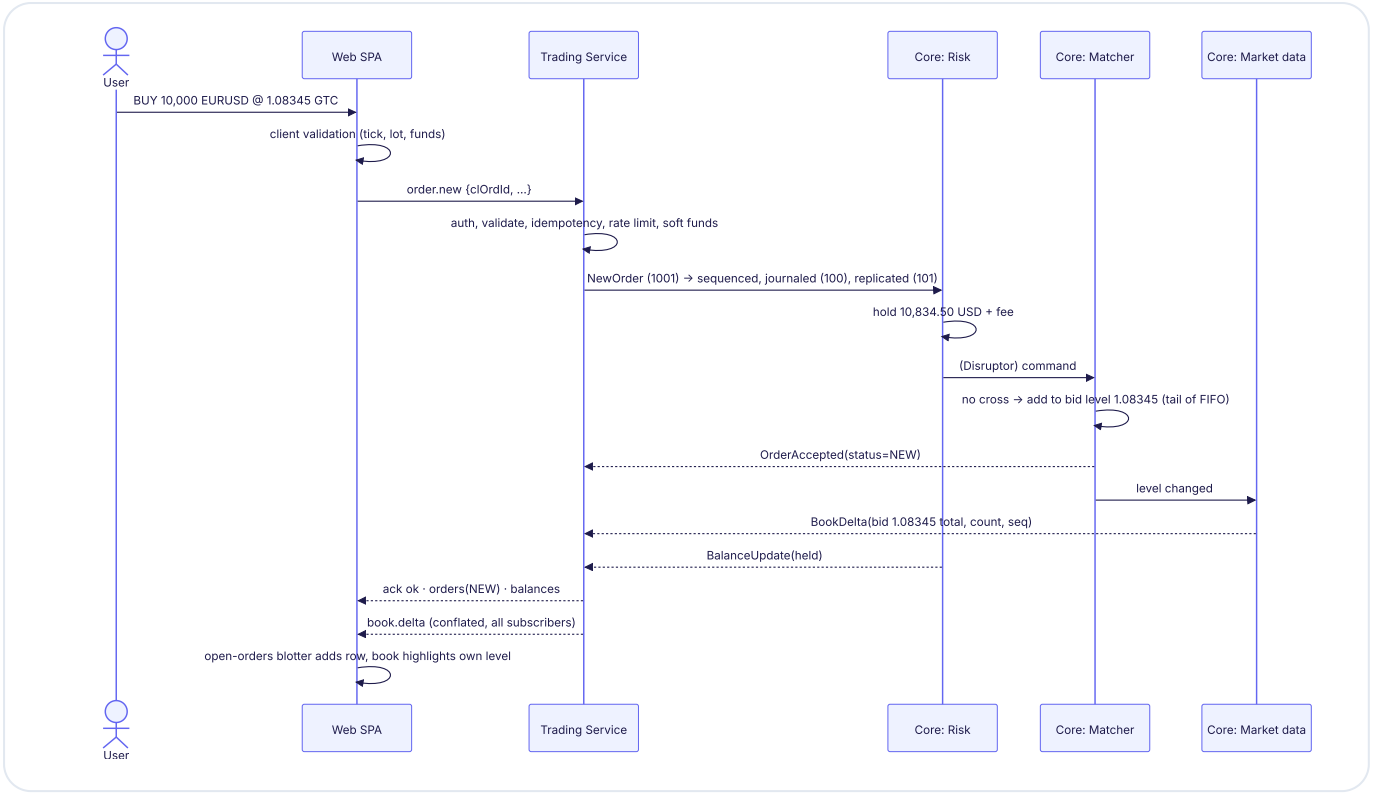
7.2 Login → live session

See Web Platform §5.4. After WS auth, the Trading Service sends `RequestUserState` if the user's state is not cached, then pushes `balances` and `orders` snapshots followed by live updates.

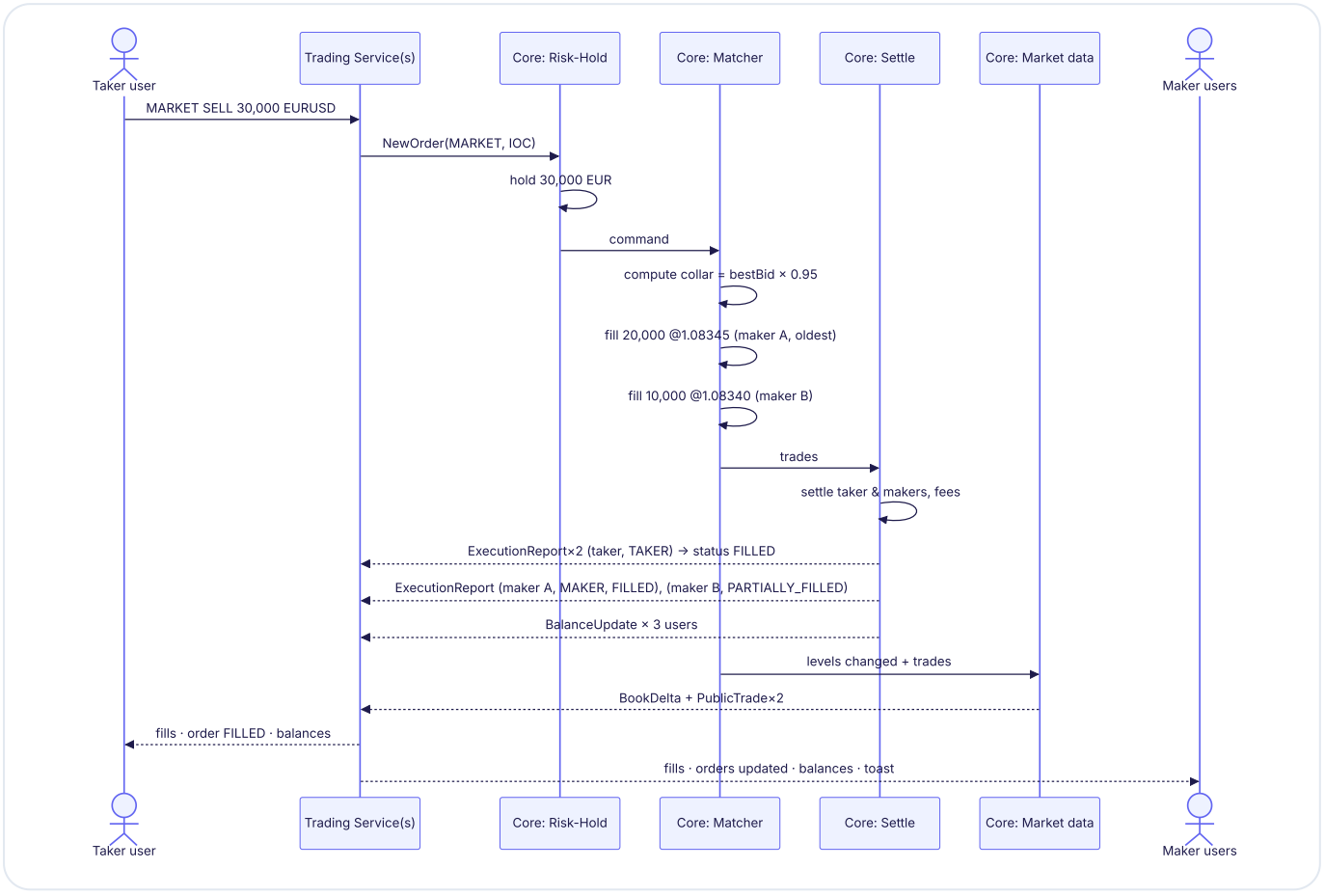
7.3 Deposit



7.4 Place limit order (rests on book)

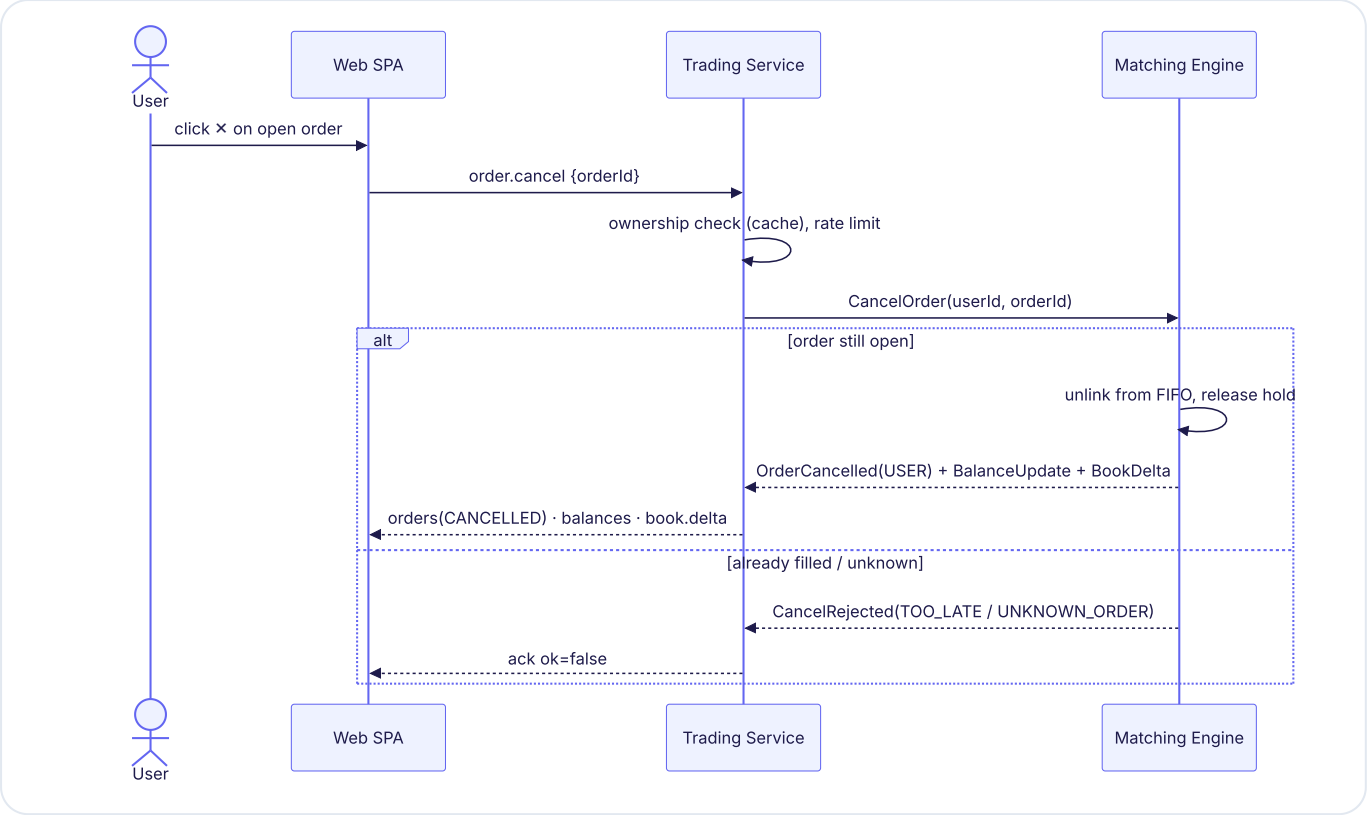


7.5 Market order matched against several makers

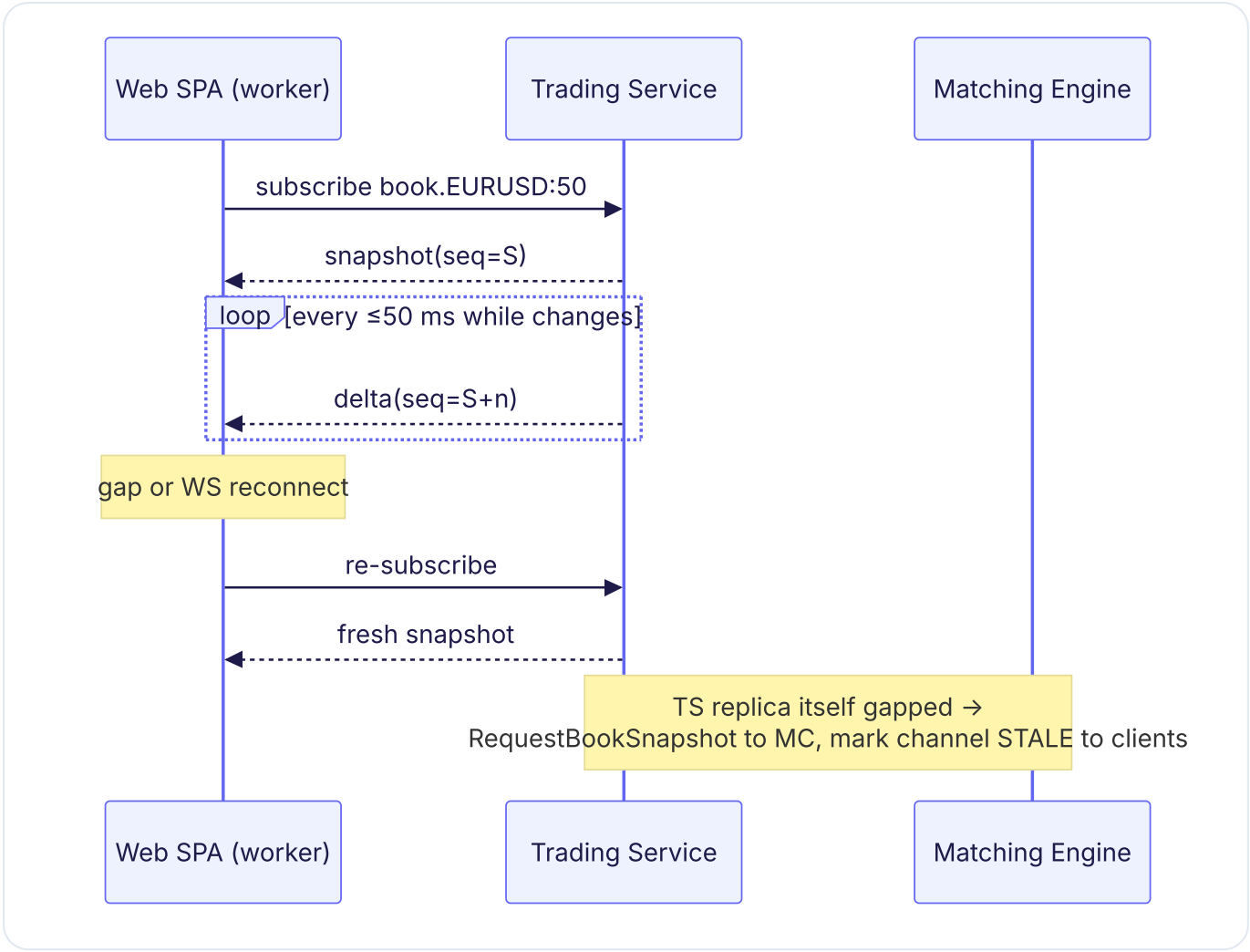


If liquidity runs out or the collar is reached, the remainder is cancelled: `OrderCancelled(reason=MARKET_NO_LIQUIDITY)` and the unused hold released.

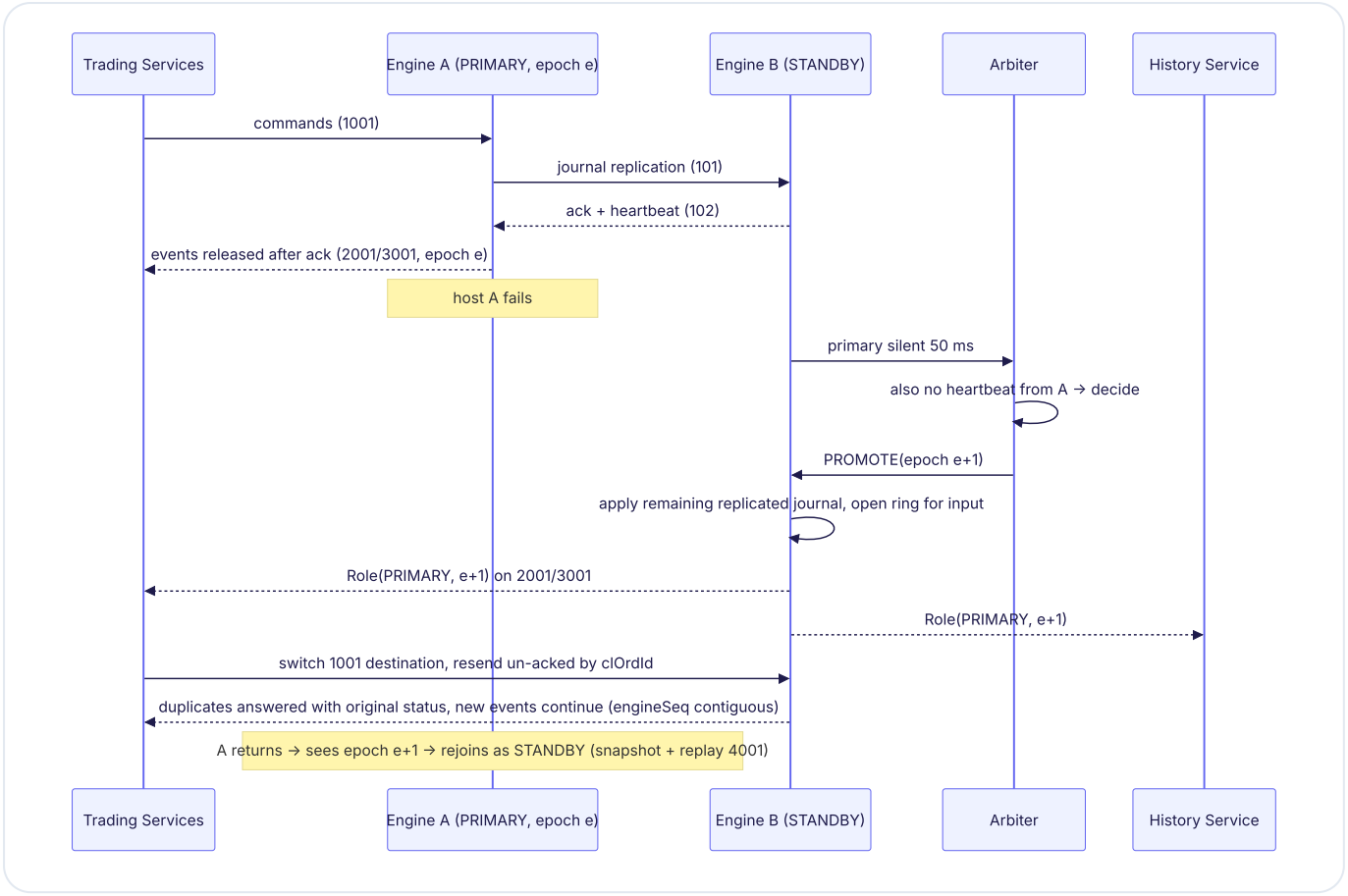
7.6 Cancel order



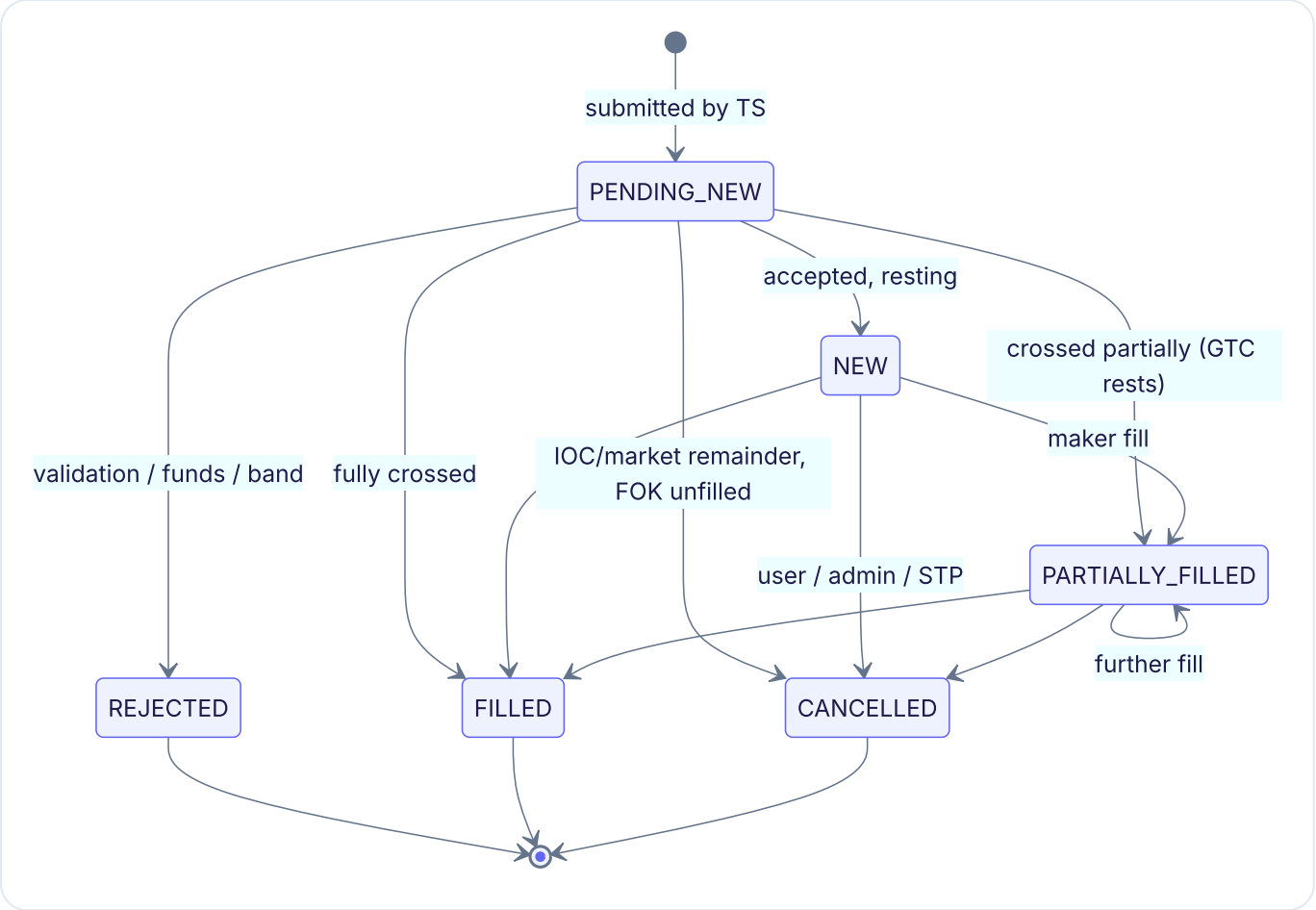
7.7 Order book subscription & resync



7.8 Engine failover (primary → standby)



7.9 Order state machine



`PENDING_NEW` exists only in the Trading Service / UI (between submit and engine response); the engine itself emits only final-of-command states.

8. Non-functional requirements, Operations & Roadmap

8.1 Performance targets

Metric	Target
Engine internal (ingress poll → egress offer)	p50 ≤ 3 µs, p99 ≤ 10 µs, p99.9 ≤ 50 µs
Trading Service in → ack out (same box as engine, IPC)	p99 ≤ 100 µs
Trading Service in → ack out (across network, single-node engine)	p99 ≤ 250 µs (10/25 GbE, kernel bypass optional)
With SYNC replication to standby (adds one core-link RTT)	p99 ≤ 150 µs same box, ≤ 300 µs across network
Browser order → ack displayed (same region)	≤ 50 ms (network dominated)
Throughput	≥ 1 M cmds/s engine; ≥ 50 k concurrent WS sessions per Trading Service instance
Market data to browser	≤ 20 updates/s per book, ≤ 100 ms staleness
Availability	99.95 % trading hours; RPO 0 (SYNC replication), RTO < 1 s failover

8.2 Security

- TLS 1.3 everywhere external; Aeron on a private, isolated network segment (VLAN); optional Aeron transport encryption not needed inside the trusted segment.
- Argon2id passwords; TOTP 2FA; account lockout & IP rate limiting; refresh-token rotation with reuse detection.
- JWT RS256/EdDSA, 5-min access tokens, JWKS rotation.
- OWASP ASVS L2 for web; CSP, CSRF protection for cookie endpoints, input validation server-side.
- PSP webhooks: signature verification + idempotency.
- Least-privilege DB roles; secrets in Vault/KMS; encrypted TOTP secrets.
- Immutable audit log (who/what/when) for admin actions, logins, funding.

8.3 Observability

- Engine: HdrHistogram per hop (ingress→risk, risk→match, match→publish), ring-buffer remaining capacity, Aeron counters (back-pressure, NAKs, retransmits, lag), snapshot duration — exported off-thread via Agrona counters → Prometheus.
- Trading Service: sessions, msg rates, conflation drop rate, reject reasons, per-hop latency.
- Business dashboards: orders/s, trades/s, volume per instrument, open orders, balances reconciliation status.

- Alerts: latency SLO breach, engine failover / replication downgrade, Redis unavailable, consumer lag, reconciliation break, PSP failures.
- Structured logs (async, off hot path) to Loki; distributed tracing on REST paths only.

8.4 Testing strategy

Level	What
Unit	Order book ops, matching rules, fee/hold arithmetic, validation
Property-based (jqwik)	Random order streams: invariants — book never crossed, FIFO respected, conservation of funds, holds ≥ 0
Determinism	Replay same log twice / on two nodes → byte-identical output
Golden-file	Scenario files (commands → expected events) shared by engine and Trading Service tests
Microbenchmarks (JMH)	add/cancel/match per op; allocation profiler must report 0 B/op on hot path
Latency harness	Open-loop load generator with coordinated-omission-correct measurement
Integration	Testcontainers (PostgreSQL, Redis) + embedded Media Driver
E2E	Playwright UI tests; simulated trader bots for soak tests
Chaos	Kill leader, drop packets, slow consumer, disk full on Archive

8.5 Operations

- Environments: dev (docker-compose, single embedded driver), test, perf (tuned hardware), prod.
- Trading sessions are 24/7 for spot; maintenance via instrument `HALTED` + snapshot + rolling upgrade (upgrade standby → switchover → upgrade old primary).
- Admin actions: halt/resume instrument, mass-cancel, suspend user, adjust fee tiers — all as engine commands (audited, replayable).
- Backups: Archive recordings replicated off-host; PostgreSQL PITR.

8.6 Delivery roadmap

Phase	Deliverables	Exit criteria
0 Foundations (1–2 wks)	Gradle multi-module repo, SBE schemas v1 + codegen, common lib (fixed-point, ids), Media Driver configs, CI, docker-compose dev stack	Schemas reviewed; build green
1 Matching Engine (4–6 wks)	Order book, matching (limit/market, GTC/IOC/FOK, post-only, STP), risk holds & settlement, Disruptor pipeline, sequencer + journal (Archive), snapshots & replay, single node, admin commands, JMH + latency harness	All invariants pass; p99 ≤ 10 µs on perf host; replay determinism proven

Phase	Deliverables	Exit criteria
2 Trading Service (3–4 wks)	Netty WS/REST, JWT auth, validation, routing, book replicas, conflation, gap recovery, user-state resync	Bot traders through WS end-to-end; resync tests
3 Identity, Funding, Persistence (3–4 wks)	Registration/login/2FA/settings; deposits with mock PSP + ledger; archive tailer → PostgreSQL; history & candles REST; reconciliation job	Deposit → trade → history visible; reconciliation zero breaks
4 Web Platform (4–6 wks, can start in parallel with 2 against a mock WS)	All screens of §5.1, themes, responsive layouts	Playwright suite green; Lighthouse ≥ 90
5 Hardening (3–4 wks)	Standby engine + replication (101/102) + arbiter, failover drills, OS/JVM tuning, security review, observability dashboards, load & soak tests	SLOs met under 2× expected peak; chaos tests pass
6 Extensions	Withdrawals, KYC tiers, API keys + FIX/REST for algos, admin console, GTD, realised P&L, price-band circuit breakers	—